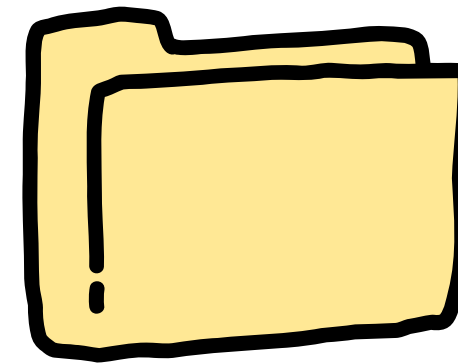
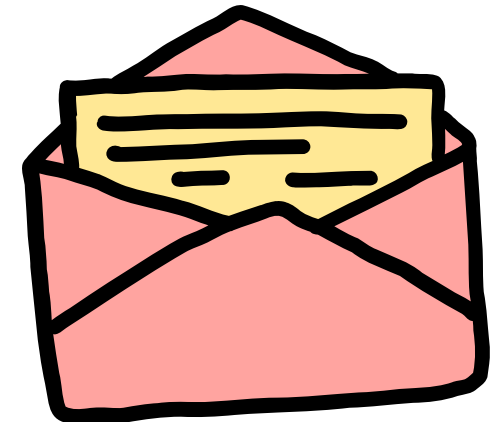


NLP BASICS

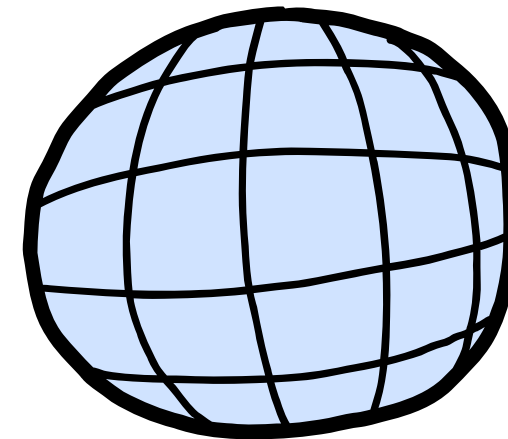
PRESENTATION



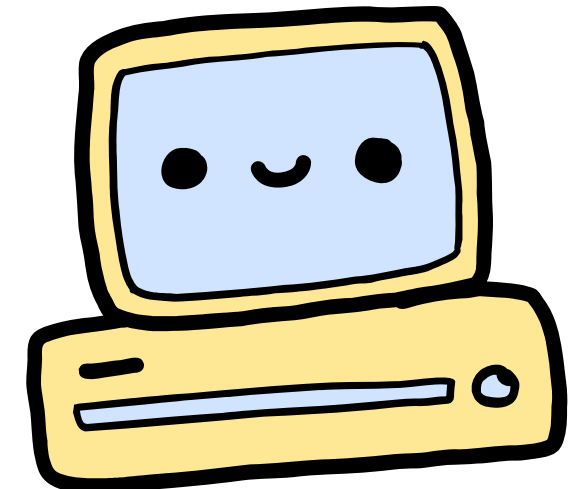
Folder



e-mail



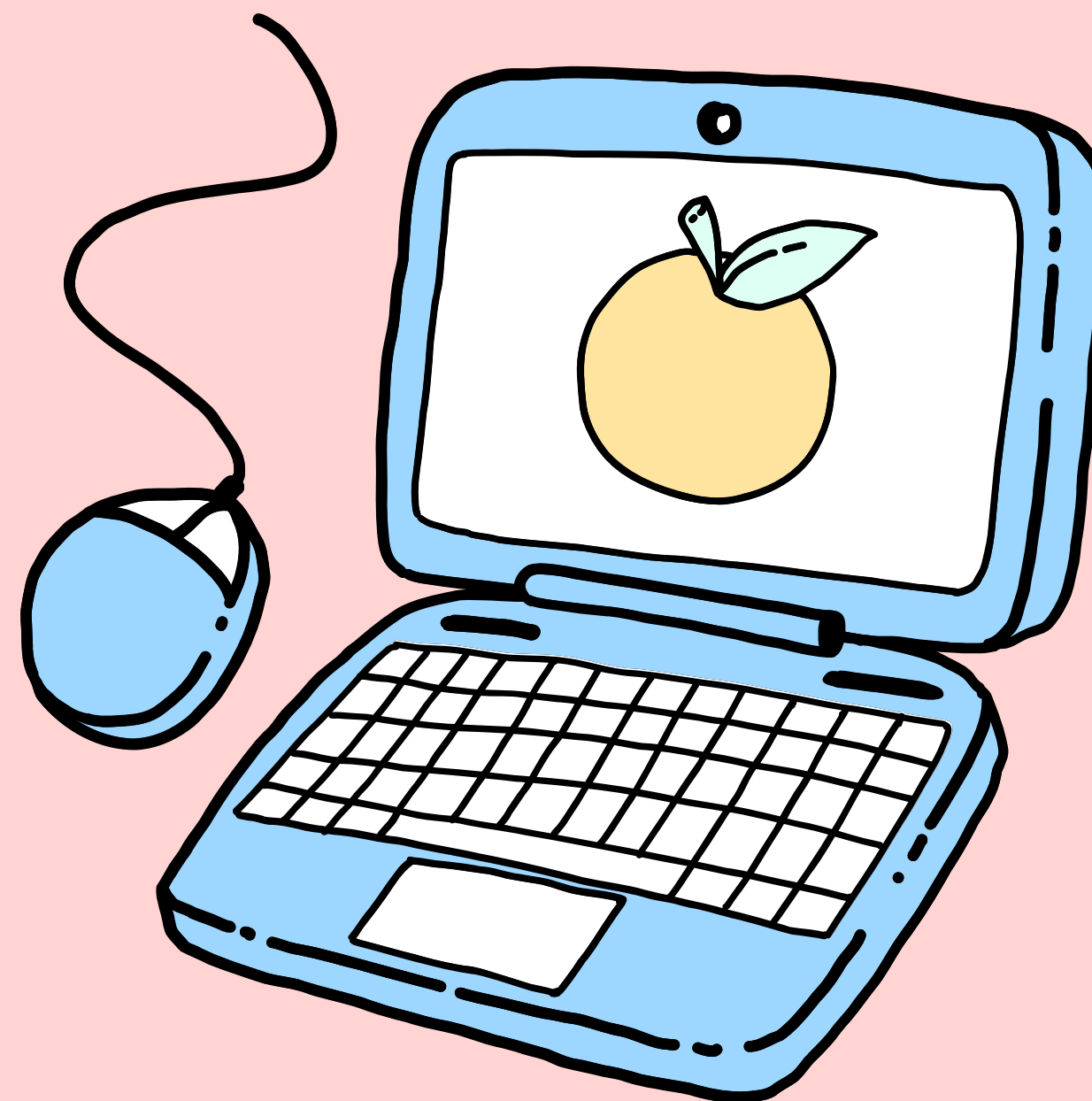
Internet



Computer

NLP

Обработка естественного языка (NLP) — это технология машинного обучения, которая дает компьютерам возможность интерпретировать, манипулировать и понимать человеческий язык.



ОСНОВНЫЕ ПОДХОДЫ

В NLP

◆ Контролируемое обучение (Supervised NLP):
Модель обучается на размеченных данных (вход–
выход).

🧠 Пример: классификация документов по темам.

◆ Неконтролируемое обучение (Unsupervised NLP):
Извлечение закономерностей из неразмеченного
текста.

💬 Пример: автозаполнение предложений на основе
контекста.

ОСНОВНЫЕ ПОДХОДЫ В NLP

- ◆ Понимание естественного языка (NLU):

Фокус на интерпретации смысла текста.

- 📌 Позволяет находить сходные значения, понимать многозначность слов.

- ◆ Генерация естественного языка (NLG):

Создание связного текста на основе данных.

- 🤖 Пример: чат-бот генерирует ответы, имитируя речь человека.

ЗАДАЧИ NLP

МАРКИРОВКА ЧАСТЕЙ РЕЧИ

Отмечает, какие слова — существительные, глаголы, прилагательные и т.п., чтобы понимать структуру предложений.

Пример:

«Кот быстро бежал по крыше»

→ [Кот/NOUN, быстро/ADV, бежал/VERB, по/ADP, крыше/NOUN]

🐱 NLP: «Окей, "кот" — это не глагол. Жаль!»

СМЫСЛ СЛОВ

Многозначные слова:

- Замок 🏰 (castle)
- Замок 🔒 (lock)

NLP решает, что ты имел в виду, исходя из контекста.
«Я сломал замок» — 🦷 не уточнил, какой.

РАСПОЗНАВАНИЕ РЕЧИ

- Превращает голос в текст
- Учитывает акценты, эмоции, ошибки

 Ты сказал: «Питон», но компьютер подумал:
«Питон — змея или язык программирования?»

МАШИННЫЙ ПЕРЕВОД

NLP помогает переводить, сохраняя смысл.

«I'm feeling blue.» → «Мне грустно», а не «Я синий».

РАСПОЗНАВАНИЕ ИМЕНОВАННЫХ СУЩНОСТЕЙ (NER)

- Кто? Где? Что?

«Джейн поехала во Францию»

→ Джейн = человек, Франция = страна

 NLP: «Ага! "Она" = "Джейн"! Я слежу за вами!»

АНАЛИЗ ТОНАЛЬНОСТИ

Определяет настроение текста:

- Позитивное 
- Негативное 
- Нейтральное 

«Обслуживание было ужасным, я в бешенстве!»

→ Эмоции: 

 NLP: «Клиент не доволен. Включаю режим извинений!»

ПРЕДОБРАБОТКА ТЕКСТА

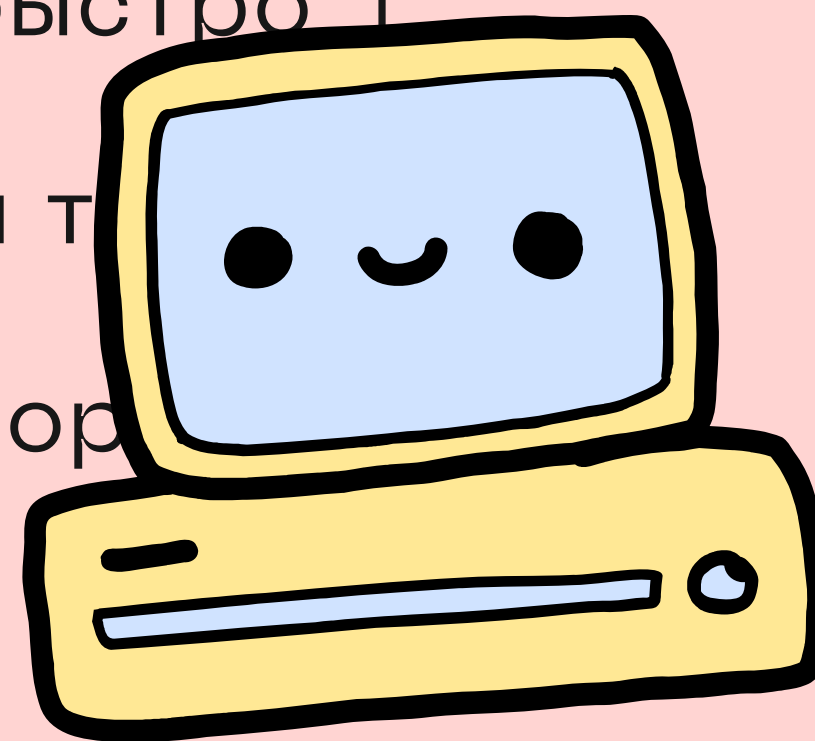
Токенизация: разбиваем текст на слова или фразы
"Кошка бежала быстро" → ["Кошка", "бежала", "быстро"]

Удаление стоп-слов: убираем "и", "в", "на", "а" и т.

Лемматизация: приводим слова к нормальной форме
"бежала" → "бежать", "кошки" → "кошка"

Стемминг: обрезаем до основы
"бегающий", "бежал", "бежит" → "бег" (но грубее чем лемматизация)

Очистка: удаление пунктуации, HTML-тегов, emoji 🎉



N-ГРАММЫ – ПАМЯТЬ МОДЕЛИ

Это последовательности из n подряд идущих слов.

Примеры (из фразы "я люблю NLP"):

- 1-граммы: ["я", "люблю", "NLP"]
- 2-граммы: ["я люблю", "люблю NLP"]
- 3-граммы: ["я люблю NLP"]

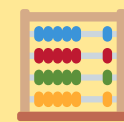


🧠 Почему это важно?

- Сохраняется контекст и порядок слов.
- Полезно для моделей, которые не видят порядок (Bag of Words, TF-IDF)

💬 "не плохо" = не то же, что "плохо" — n -граммы это улавливают!

FEATURE ENGINEERING



Статистические признаки:

- Длина текста
- Кол-во знаков препинания (! ?)
- Частота заглавных букв
- Количество emoji или ссылок



TF (Term Frequency):

- Сколько раз слово встречается в тексте

FEATURE ENGINEERING

 TF-IDF (Term Frequency – Inverse Document Frequency):

- Частое в этом тексте + редкое в других → ВАЖНО

 n-граммы как признаки:

- 2-граммы из фразы: "чёрный кофе горячий"
- → "чёрный кофе", "кофе горячий"

 Признак: если "кошки + мурлыкают" → это ПОЗИТИВНЫЙ ОТЗЫВ!

ВЕКТОРНЫЕ ПРЕДСТАВЛЕНИЯ ТЕКСТА

BAG OF WORDS (BOW)

считаем, сколько раз каждое слово встретилось

"кошка любит рыбу" → {"кошка": 1, "любит": 1, "рыбу": 1}

+

- Просто
- Работает на маленьких датасетах

–

- Не учитывает порядок
- Слова "кошка рыбу любит" → то же, что и "рыбу любит кошка"
- Вектор может быть огромным (100 тыс. слов = 100 тыс. признаков)

🍷 BoW: «Каждое слово по кирпичику — а смысл

неважен!» • • •



TF-IDF

усиливает редкие, но важные слова

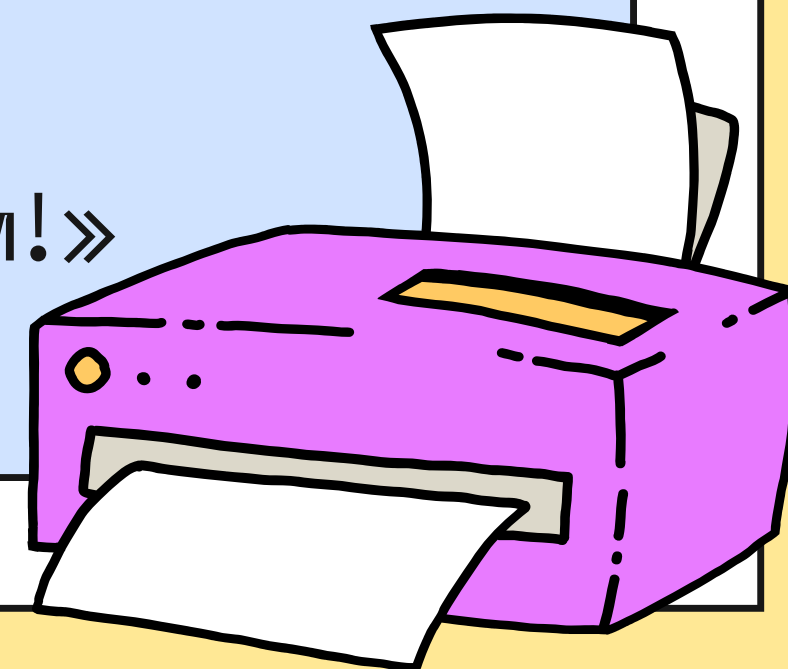
- "банк" важен в теме экономики
- "и", "на", "в" — не важны нигде

TF-IDF = частота слова в документе × логарифм
(обратной частоты в корпусе)

- "пельмени" 10 раз в 1 документе, но редко в других → высоко
- "еда" 1000 раз везде → низко



TF-IDF: «Слово редкое? Тогда это не просто шум!»



ЭМБЕДДИНГИ

"умные" векторы, обученные на больших корпусах с пониманием контекста



Вместо:

⌈ "мама": 0, "мыла": 1, "раму": 2, ... ⌋

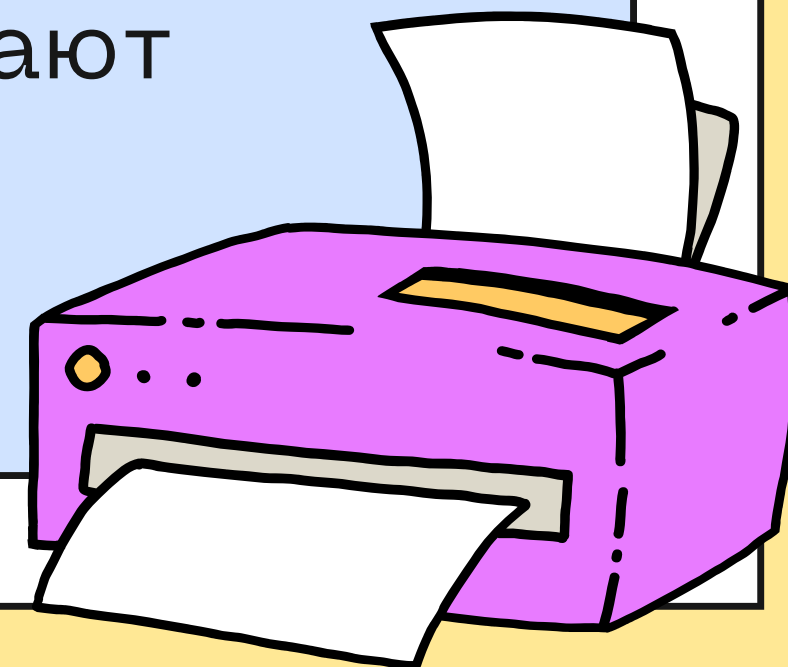
→ Получаем:

"мама" → [0.32, 0.67, -0.11, ...]

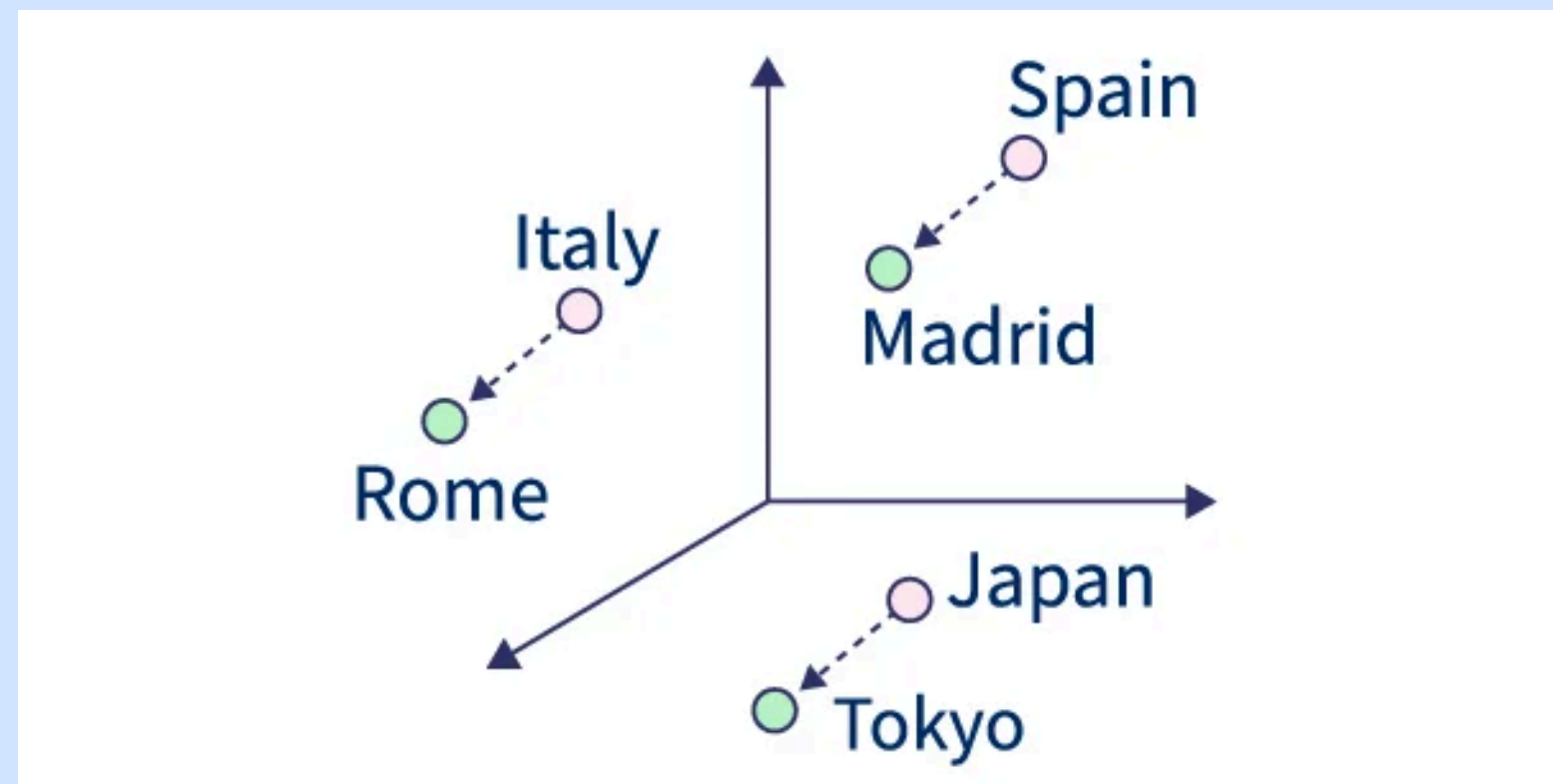
Слова, встречающиеся в похожих контекстах, получают похожие векторы:

- "кот" ≈ "кошка"

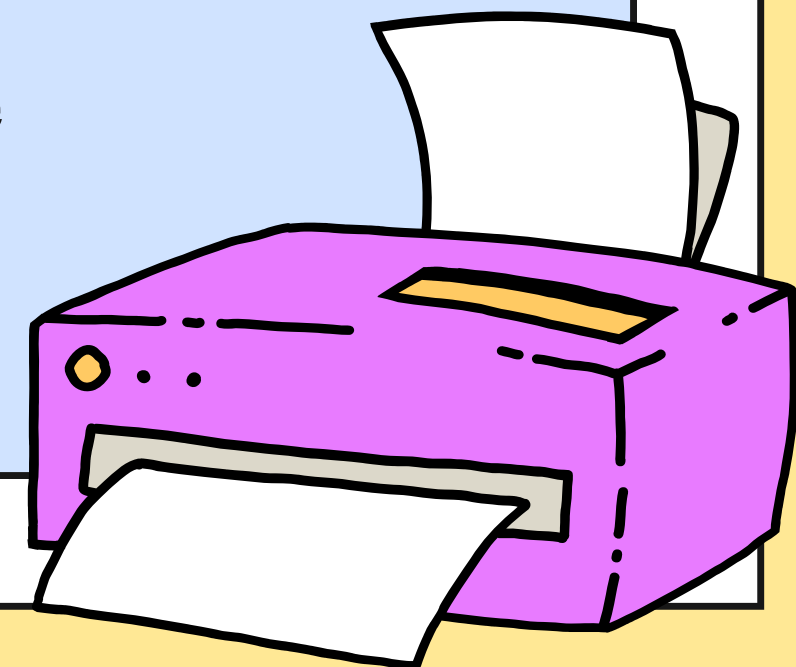
- "король" - "мужчина" + "женщина" = "королева"



ЭМБЕДДИНГИ



🎯 Эмбеддинг = точка в многомерной вселенной, где близость → семантическая близость.



WORD2VEC

Идея: учим модель так, чтобы:

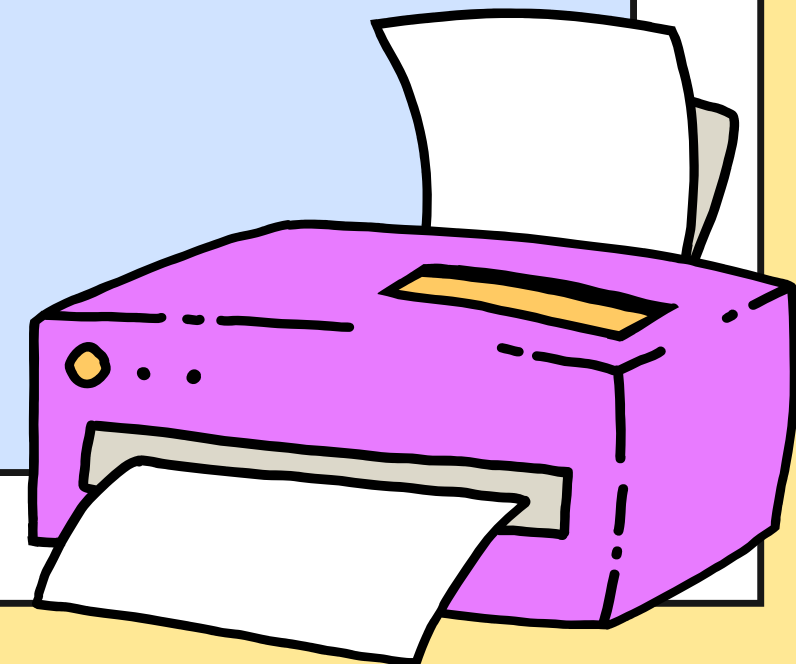
- по контексту предсказывать слово (CBOW)
- по слову предсказывать контекст (Skip-gram)

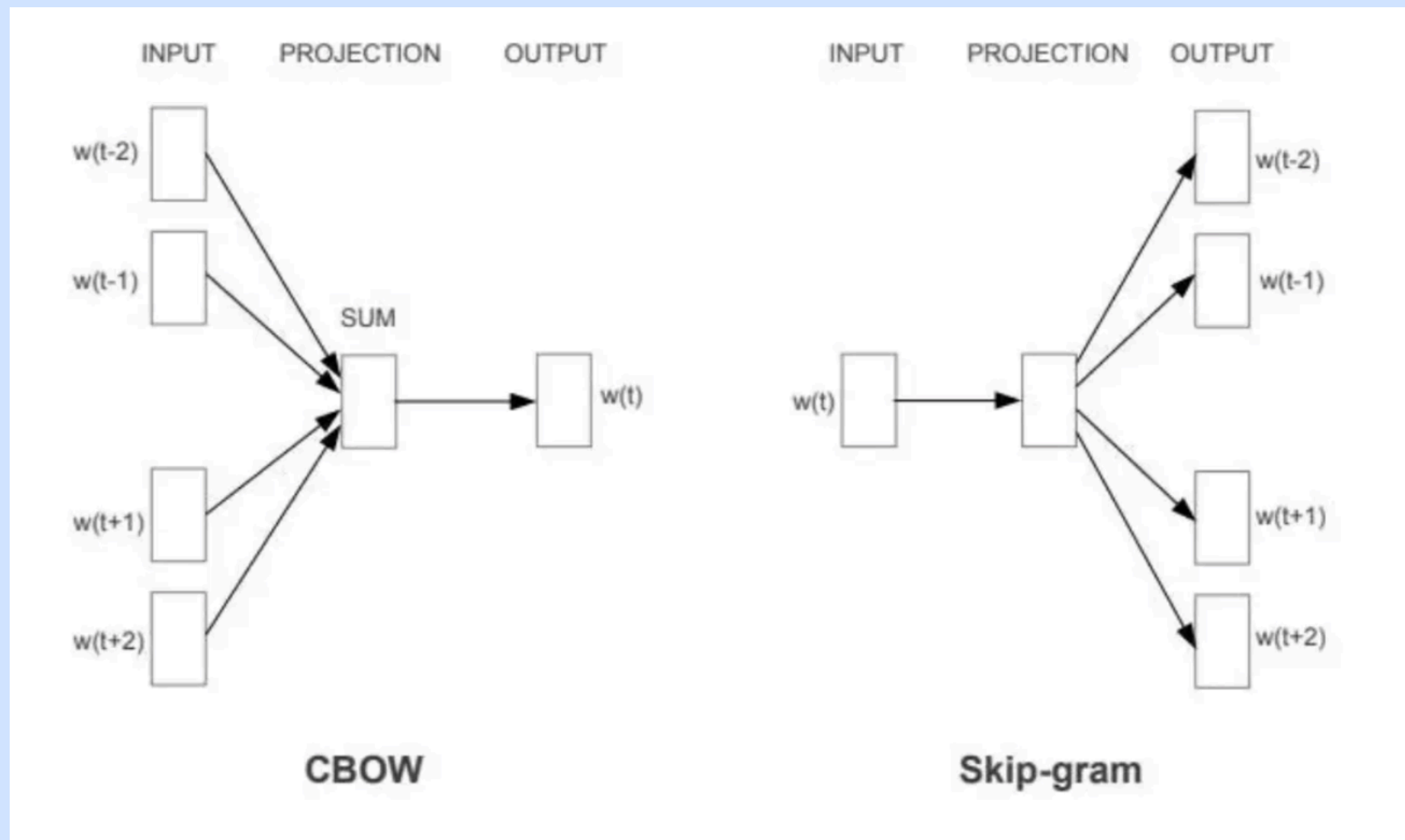
📖 Пример:

“Кошка лежит на диване”

Слово “лежит” окружено “Кошка”, “на”, “диване”

↻ Обновляем векторы слов так, чтобы они были полезны для предсказания.





классная статья!

GLOVE (GLOBAL VECTORS)

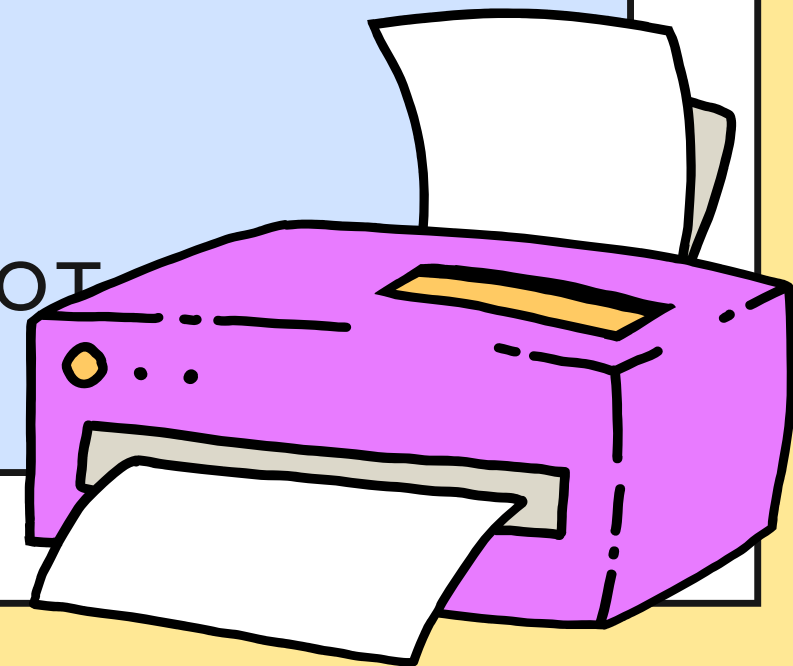
В отличие от Word2Vec, GloVe:

- не учится на последовательности
- строит матрицу соотношений слов во всех текстах:
 - насколько часто слово i встречается рядом со словом j

🧩 Пример:

- "кошка" и "мяукать" → часто вместе
- "кошка" и "чайник" → почти не встречаются

⚖️ GloVe строит вектора так, чтобы пропорции частот сохранялись.



GLOVE (GLOBAL VECTORS)

$w \in \mathbb{R}^d$ are word vectors

probe word

$$F(w_i, w_j, \tilde{w}_k) = \frac{P_{ik}}{P_{jk}}$$

co-relations between the word w_i and w_j

co-occurrence probabilities for the word w_j and w_k

ещё классная статья! <https://jonathan-hui.medium.com/nlp-word-embedding-glove-5e7f523999f6>

GLOVE (GLOBAL VECTORS)

Probability and Ratio	$k = \text{solid}$	$k = \text{gas}$	$k = \text{water}$	$k = \text{fashion}$
$P(k \text{ice})$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \text{steam})$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k \text{ice})/P(k \text{steam})$	8.9	8.5×10^{-2}	1.36	0.96

Very small or large:

solid is related to ice but not steam, or
gas is related to steam but not ice

close to 1:

water is highly related to ice and steam, or
fashion is not related to ice or steam.

ИДА

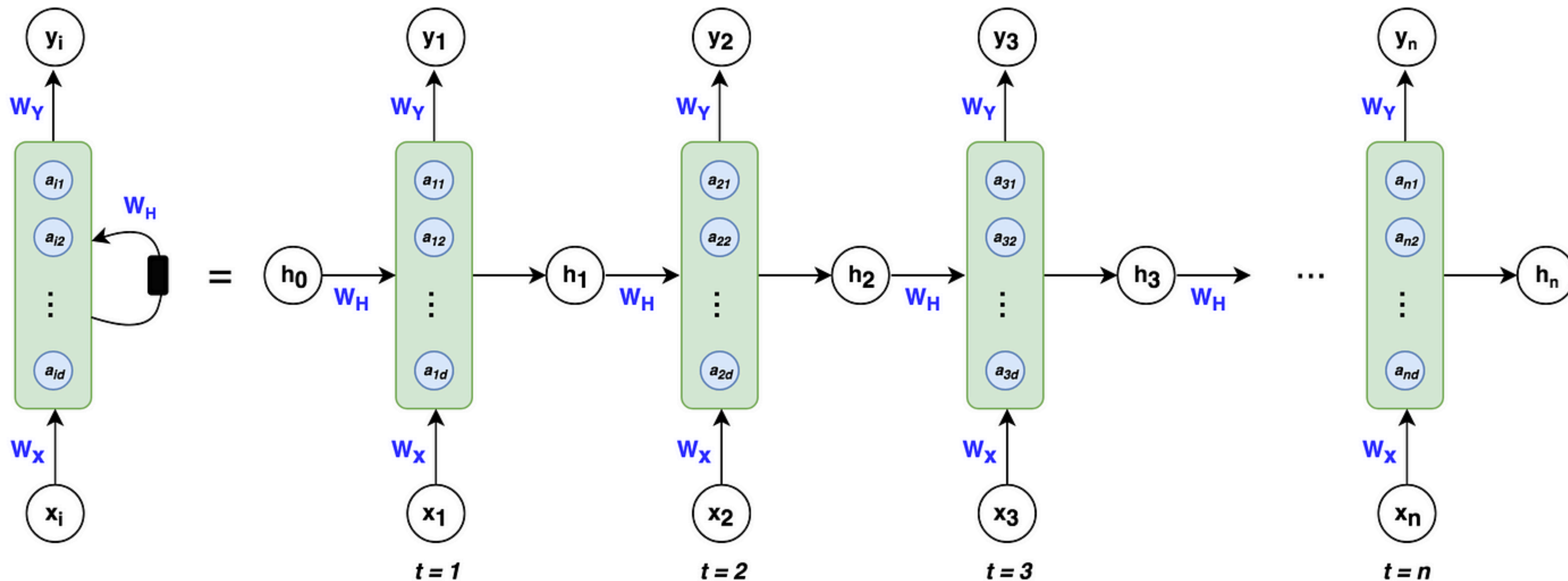
МЫ ПОЛУЧИЛИ ЧИСЛОВЫЕ
ВЕКТОРА. ЧТО ДАЛЬШЕ?

💡 Эмбеддинги превращают текст в топливо для ML и DL моделей.

- ◆ Сравниваем → семантика (поиск, антонимы, синонимы)
- ◆ Классифицируем → тональность, спам, темы
- ◆ Генерируем → перевод, диалоги, резюме
- ◆ Кластеризуем → тематики, группы слов
- ◆ Размечаем → сущности, части речи

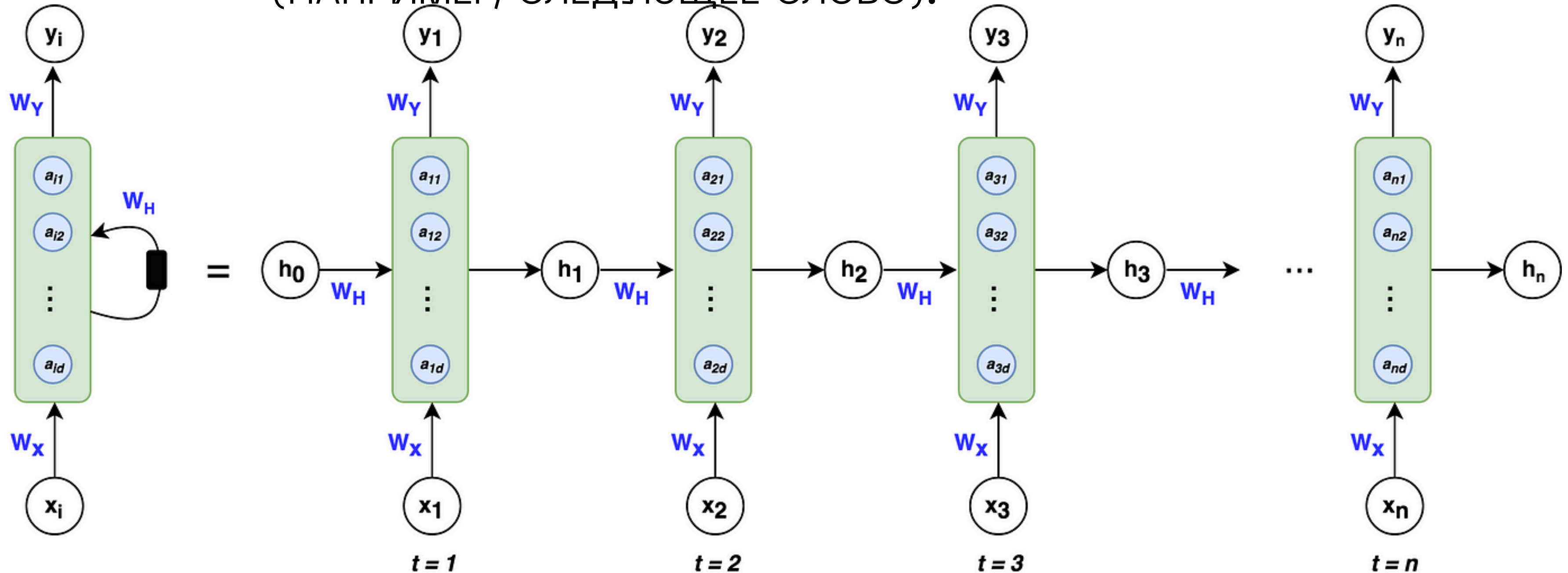
RNN

ПОМНИТ ТО, ЧТО ВИДЕЛА РАНЬШЕ.
ПРЕДСТАВЬ, ЧТО RNN — ЭТО ЧЕЛОВЕК, КОТОРЫЙ ЧИТАЕТ ТЕКСТ
ПО СЛОВАМ. КАЖДОЕ НОВОЕ СЛОВО ОН ПОНИМАЕТ ЛУЧШЕ, ЕСЛИ
ПОМНИТ ПРЕДЫДУЩЕЕ.

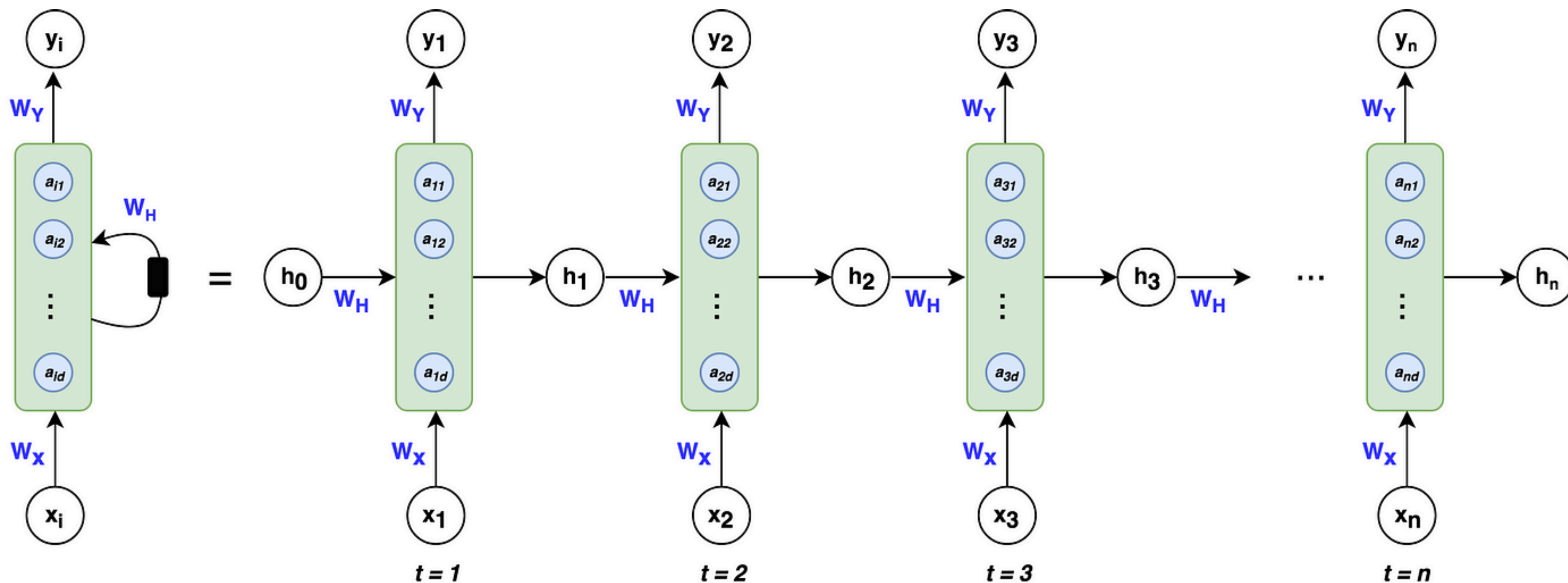


RNN

- НА КАЖДОМ ШАГЕ СЕТЬ ПОЛУЧАЕТ ОДНО СЛОВО (ВХОД).
- ОНА ЗАПОМИНАЕТ, ЧТО БЫЛО ДО ЭТОГО, И ДОБАВЛЯЕТ НОВУЮ ИНФОРМАЦИЮ.
- ЭТО ПОМОГАЕТ ЕЙ ПРЕДСКАЗЫВАТЬ, ЧТО БУДЕТ ДАЛЬШЕ (НАПРИМЕР, СЛЕДУЮЩЕЕ СЛОВО).

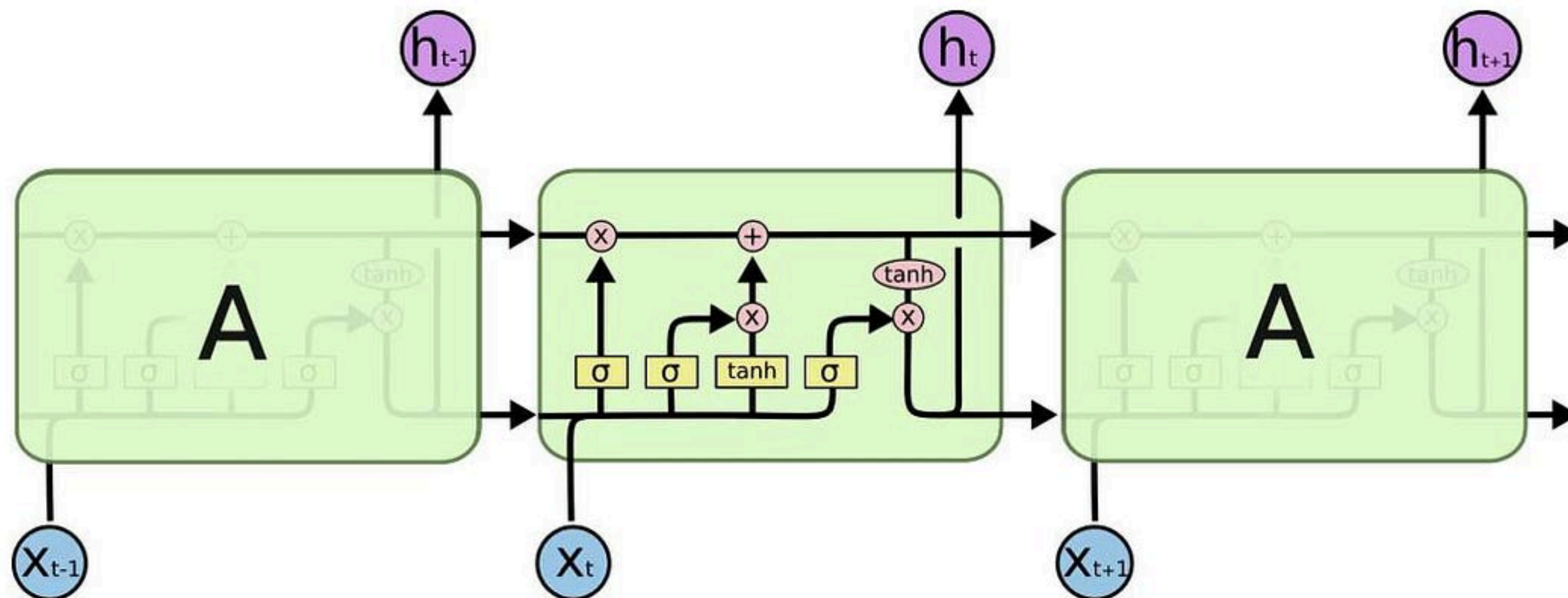


- $X_1, X_2, \dots, X_n$ — ЭТО ВХОДЫ (НАПРИМЕР, СЛОВА ИЛИ БУКВЫ).
- КАЖДЫЙ ЗЕЛЁНЫЙ БЛОК — ЭТО НЕЙРОННЫЙ СЛОЙ, КОТОРЫЙ ОБРАБАТЫВАЕТ ВХОД И ПАМЯТЬ ИЗ ПРОШЛОГО (H_{t-1}).
- 🔄 ПАМЯТЬ ПЕРЕДАЁТСЯ ДАЛЬШЕ — КАК ЕСЛИ БЫ ЧИТАТЕЛЬ ЗАПОМИНАЛ ПРОЧИТАННОЕ.
- $Y_1, Y_2, \dots, Y_n$ — ЭТО ОТВЕТЫ СЕТИ, КОТОРЫЕ ОНА ВЫДАЁТ НА КАЖДОМ ШАГЕ.
- 🔧 ВНУТРИ БЛОКОВ ИСПОЛЬЗУЮТСЯ "ВЕСА" (W_x, W_h, W_y), КОТОРЫЕ ПОМОГАЮТ ПРАВИЛЬНО КОМБИНИРОВАТЬ ВХОД И ПАМЯТЬ, ЧТОБЫ СЕТЬ УЧИЛАСЬ.

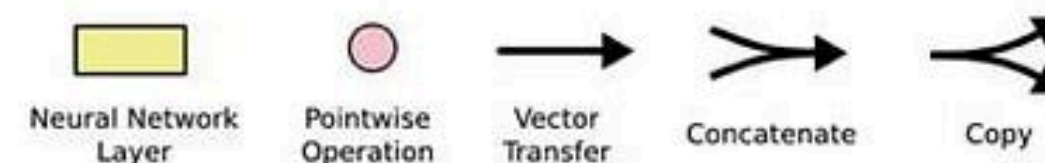


LSTM

"ДЛИННАЯ КРАТКОСРОЧНАЯ ПАМЯТЬ" — ОСОБЕННЫЙ ТИП НЕЙРОСЕТИ, КОТОРАЯ НЕ ПРОСТО ЗАПОМИНАЕТ, А ЗНАЕТ, ЧТО ВАЖНО ЗАПОМНИТЬ, А ЧТО — ЗАБЫТЬ.



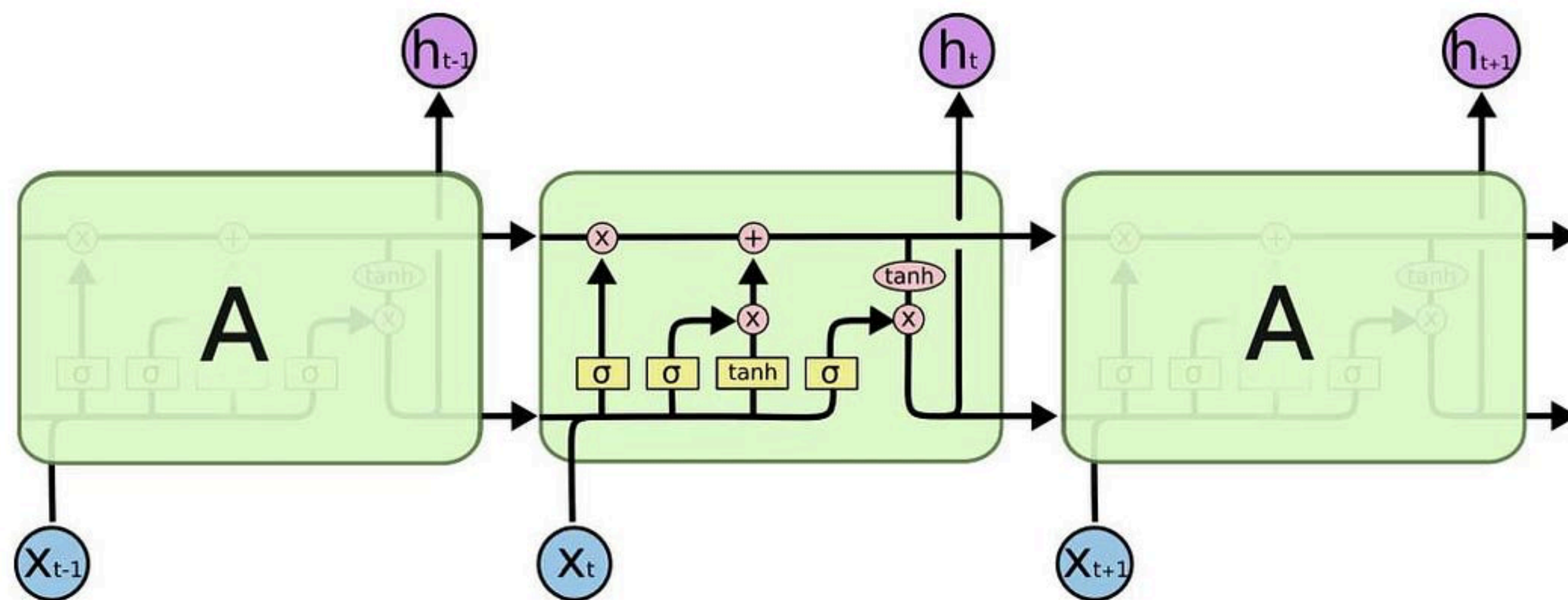
long-short term memory modules used in an RNN



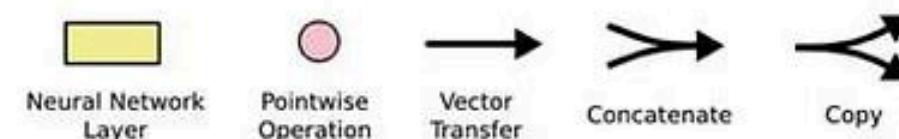
LSTM

ПРЕДСТАВЬТЕ СЕБЕ ШКОЛЬНИКА, КОТОРЫЙ:

- ВЕДЁТ ДВЕ ТЕТРАДИ — ОДНУ ДЛЯ ВАЖНЫХ ЗАМЕТОК, ДРУГУЮ ДЛЯ ТЕКУЩИХ ДЕЛ,
- КАЖДЫЙ РАЗ РЕШАЕТ:
- «ЧТО ОСТАВИТЬ НА ПАМЯТЬ НАДОЛГО, А ЧТО НУЖНО ТОЛЬКО СЕЙЧАС?»
- И ПРИ ЭТОМ ФИЛЬТРУЕТ, ЧТО СТЕРЕТЬ, ЧТО ЗАПИСАТЬ, А ЧТО РАССКАЗАТЬ ВСЛУХ.



long-short term memory modules used in an RNN

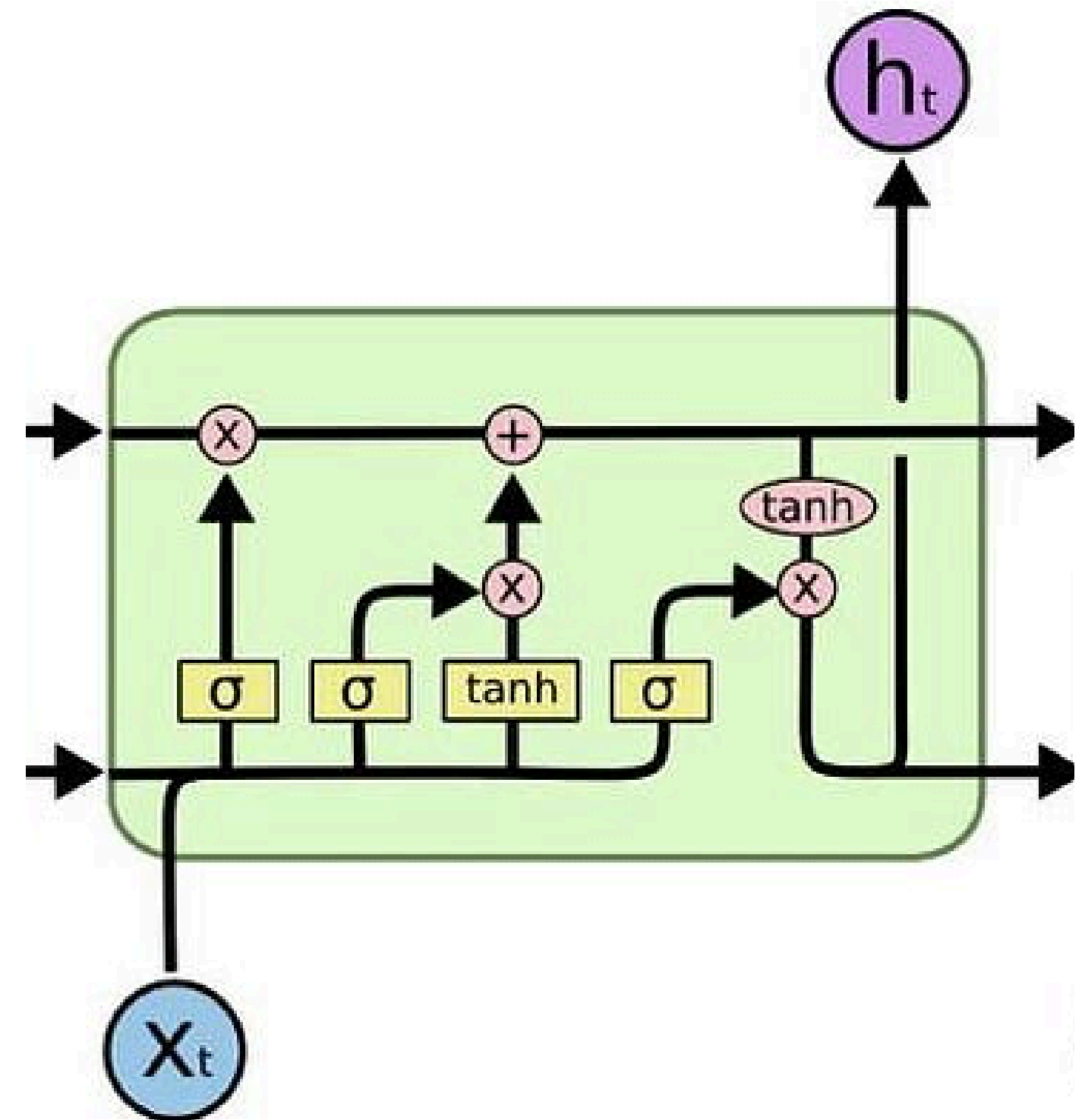


LSTM

● X_t — ЭТО НОВЫЙ ВХОД (НАПРИМЕР, СЛОВО).
● H_{t-1} — ЭТО "ПАМЯТЬ" ИЗ ПРОШЛОГО ШАГА.
● ЗЕЛЁНЫЙ БЛОК — ЭТО МОЗГ, КОТОРЫЙ РЕШАЕТ:

- 🔓 ЧТО ЗАБЫТЬ? (FORGET GATE)
- 📥 ЧТО ЗАПОМНИТЬ? (INPUT GATE)
- 🧠 ЧТО ОСТАВИТЬ В ПАМЯТИ? (CELL)
- 📤 ЧТО ОТДАТЬ ДАЛЬШЕ? (OUTPUT GATE)

🧱 СИМВОЛЫ:
СИГМОИДА — РЕШАЕТ "ДА ИЛИ НЕТ" (НАПРИМЕР, ЗАПОМНИТЬ 90% ИЛИ 10%).
ТАНН — ПОМОГАЕТ СОХРАНИТЬ ИНФОРМАЦИЮ "В ПРАВИЛЬНЫХ ГРАНИЦАХ".
× И + — УМНОЖЕНИЯ И СЛОЖЕНИЯ, КАК В АРИФМЕТИКЕ.



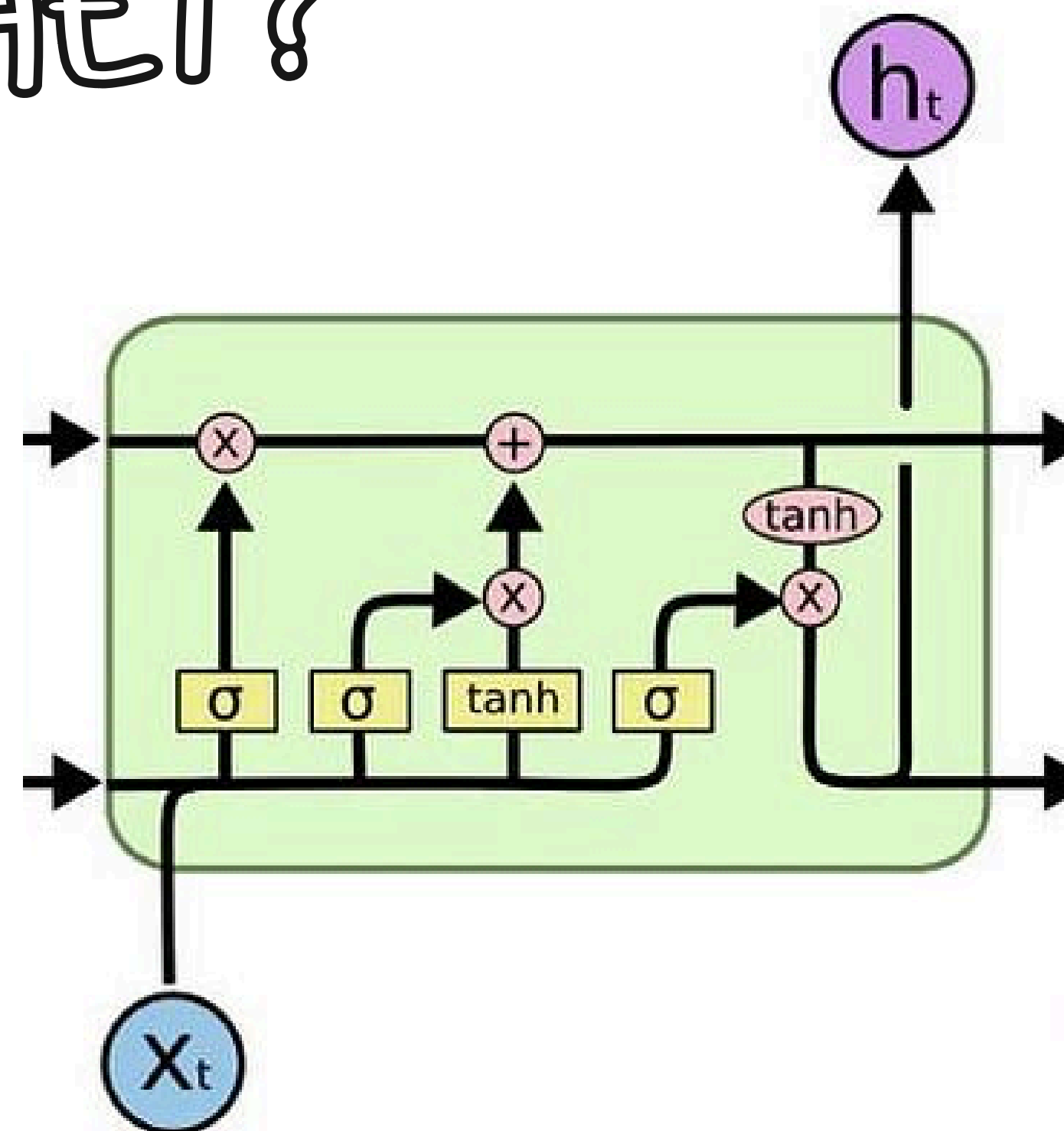
ГДЕ LSTM ЗАБЫВАЕТ?

♦ "FORGET GATE" — ЭТО ПЕРВЫЙ УЧАСТОК, ГДЕ ПРИНИМАЕТСЯ РЕШЕНИЕ, ЧТО УДАЛИТЬ ИЗ ПАМЯТИ.

НА КАРТИНКЕ ОН НАХОДИТСЯ СЛЕВА, ГДЕ ВИДНО:

- СИМВОЛ СИГМОИДА,
- СТРЕЛКА ИДЁТ ОТ ВХОДОВ x_t И h_{t-1} ,
- ЗНАК $\times$ (УМНОЖЕНИЕ),
- ЭТО ВЛИЯЕТ НА СТАРУЮ ПАМЯТЬ C_{t-1} .

📌 ЭТО ОЗНАЧАЕТ: МЫ БЕРЁМ СТАРУЮ ПАМЯТЬ И УМНОЖАЕМ НА ЧИСЛО ОТ 0 ДО 1, ЧТОБЫ ЧАСТИЧНО ИЛИ ПОЛНОСТЬЮ ЗАБЫТЬ.

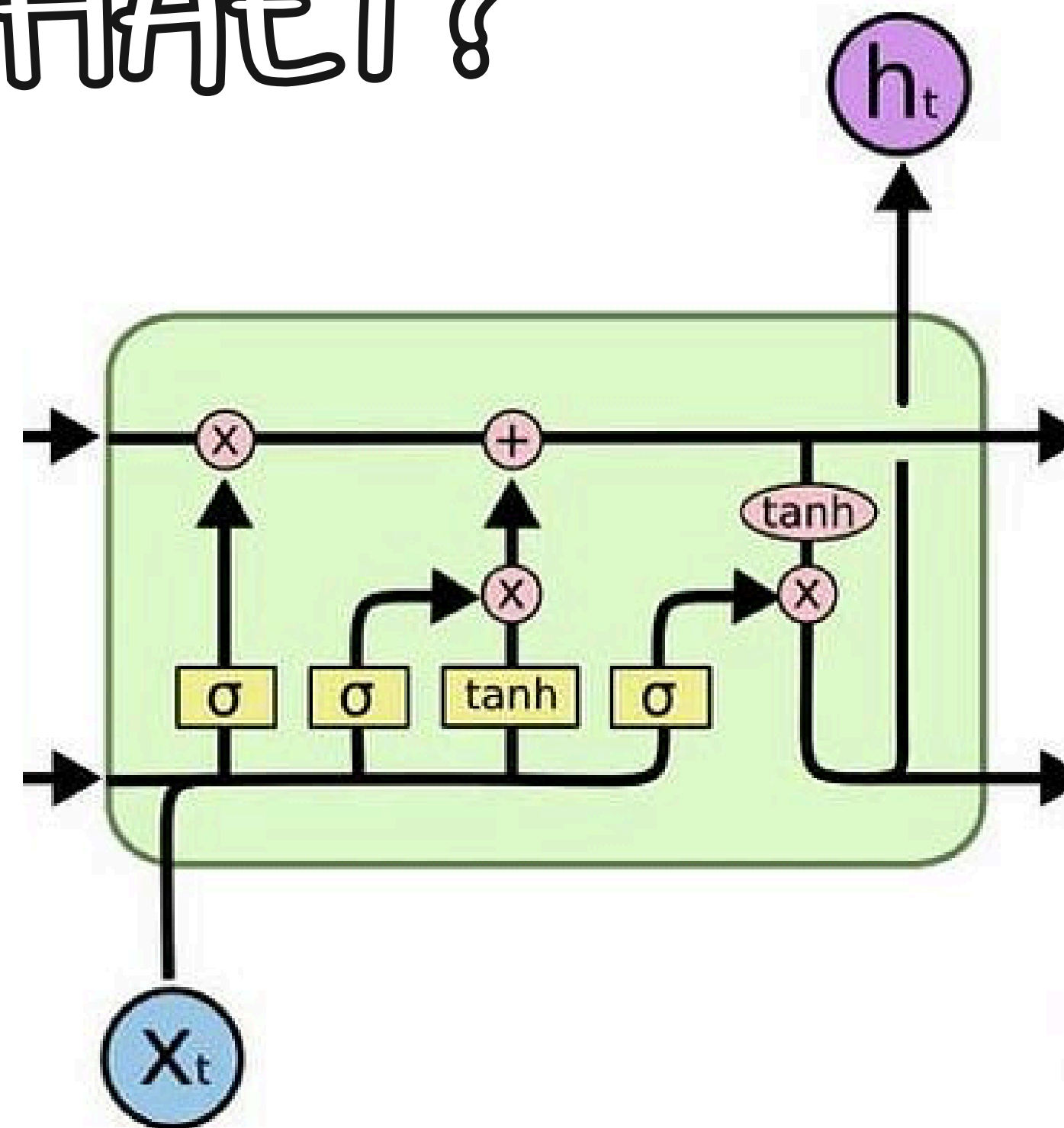


ГДЕ LSTM ЗАПОМИНАЕТ?

◆ "INPUT GATE" + "CANDIDATE" — СЛЕДУЮЩИЙ УЧАСТОК, ГДЕ LSTM РЕШАЕТ, ЧТО НОВОГО СТОИТ ЗАПОМНИТЬ.

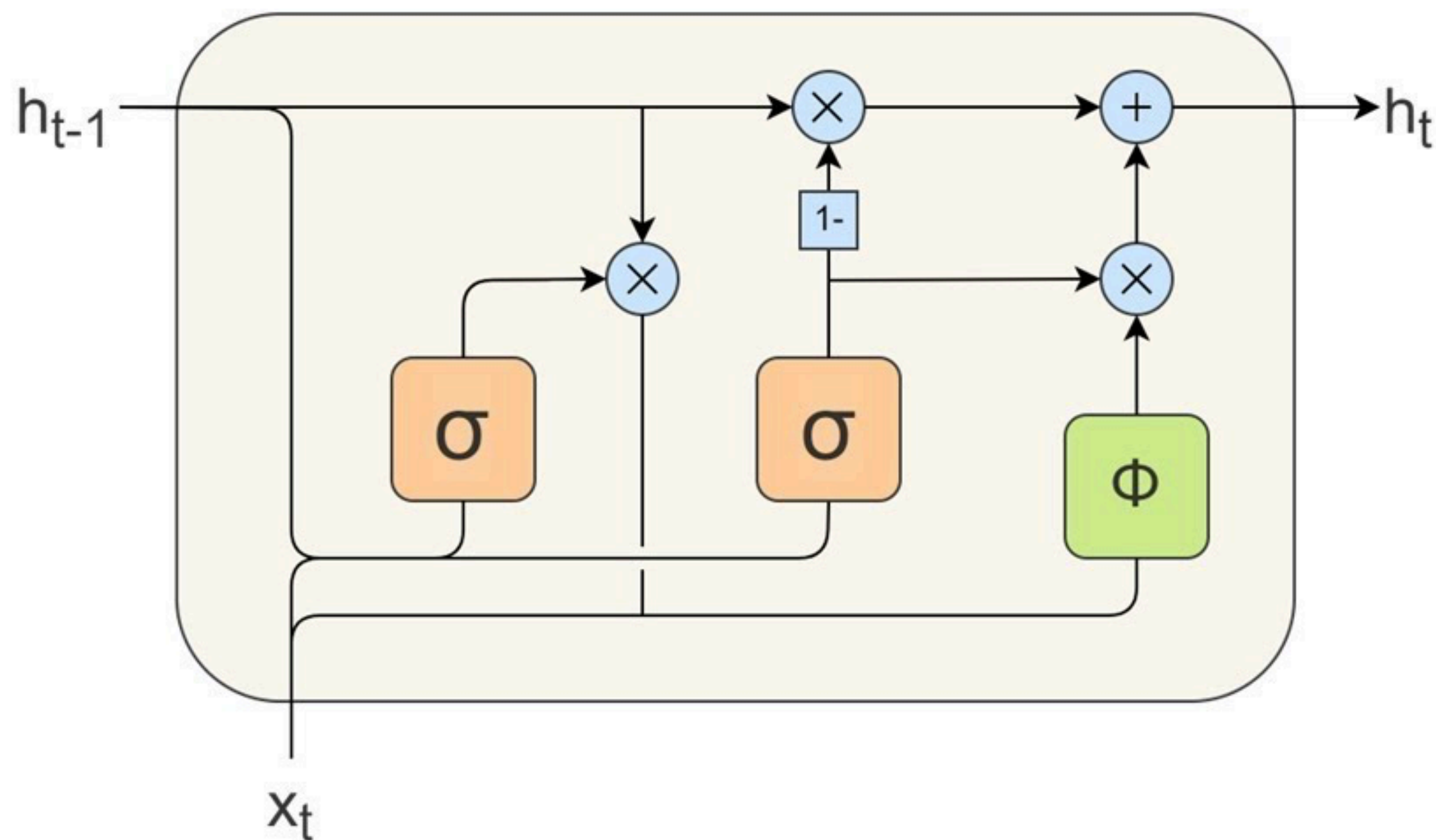
- ВХОД x_t И h_{t-1} СНОВА ИДУТ ЧЕРЕЗ:
 - ЕЩЁ ОДНУ σ (РЕШАЕТ "ДА/НЕТ", СТОИТ ЛИ СОХРАНЯТЬ),
 - И $\tanh$, КОТОРЫЙ СОЗДАЁТ НОВУЮ ИНФОРМАЦИЮ, КОТОРУЮ МОЖНО ДОБАВИТЬ,
- ЗАТЕМ СНОВА ЗНАК $\times$ И ЗНАК $+$ — ЭТО ЗНАЧИТ НОВОЕ $+$ ОТФИЛЬТРОВАННАЯ СТАРАЯ ПАМЯТЬ = НОВАЯ ПАМЯТЬ C_t .

📌 ТО ЕСТЬ: НОВАЯ ВАЖНАЯ ИНФОРМАЦИЯ ДОБАВЛЯЕТСЯ В ПАМЯТЬ.



GRU

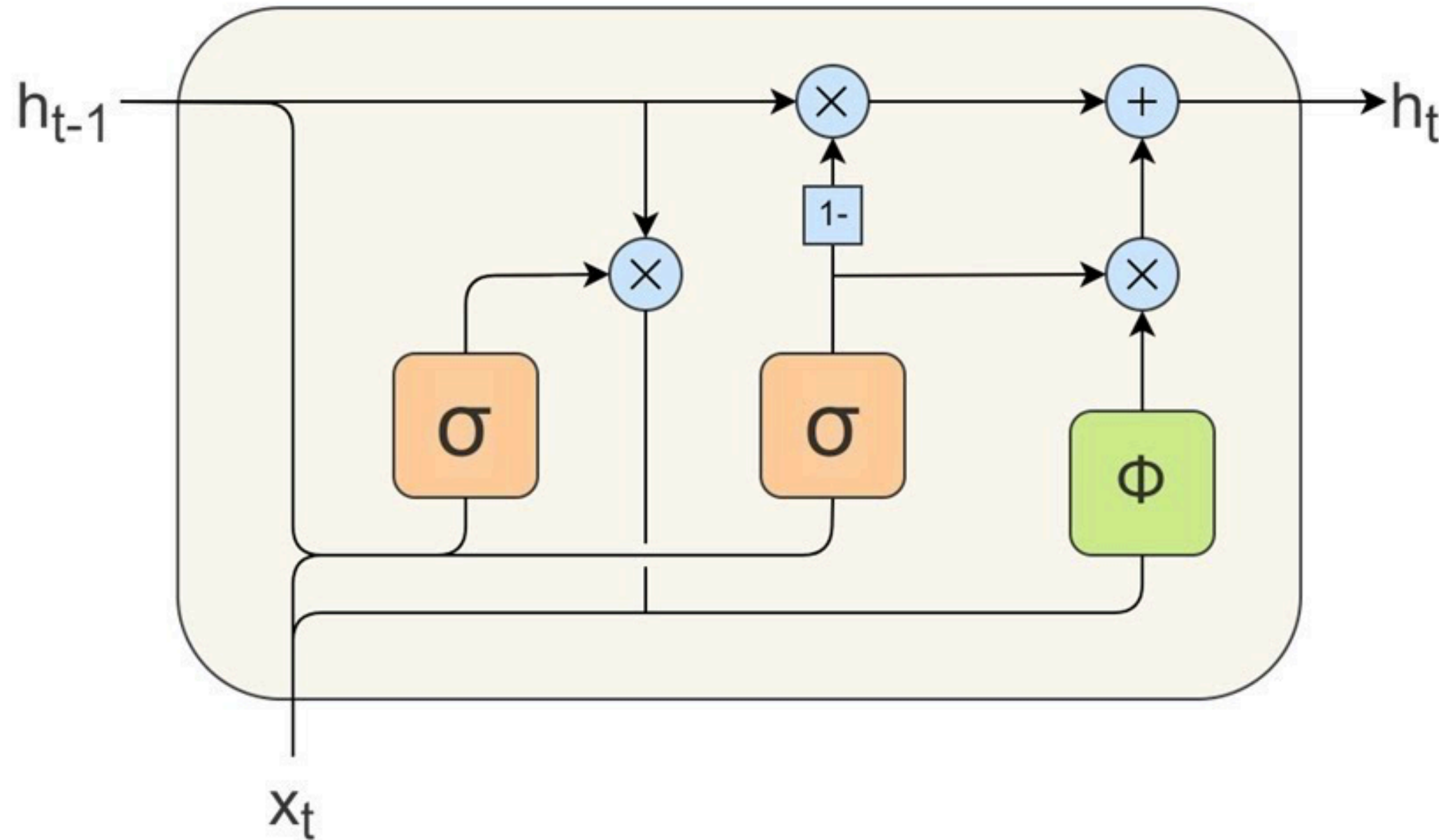
"УМНАЯ НЕЙРОСЕТЬ", КОТОРАЯ РЕШАЕТ САМА, ЧТО
ЗАПОМНИТЬ, А ЧТО ЗАБЫТЬ.
РАБОТАЕТ КАК LSTM, НО ПРОЩЕ И БЫСТРЕЕ.



GRU

ПРЕДСТАВЬТЕ СЕБЕ ШКОЛЬНИКА, КОТОРЫЙ:

- ВЕДЁТ ОДНУ УМНУЮ ТЕТРАДЬ, КУДА ПИШЕТ ТОЛЬКО ТО, ЧТО ДЕЙСТВИТЕЛЬНО ПОЛЕЗНО,
- БЫСТРО РЕШАЕТ:
- «СТАРОЕ ЕЩЁ НУЖНО? ИЛИ МОЖНО ОБНОВИТЬ ЧЕМ-ТО ПОНОВЕЕ?»
- И НЕ ТРАТИТ ВРЕМЯ НА ЛИШНИЕ ЗАПИСИ — ВСЁ ПО ДЕЛУ.



GRU

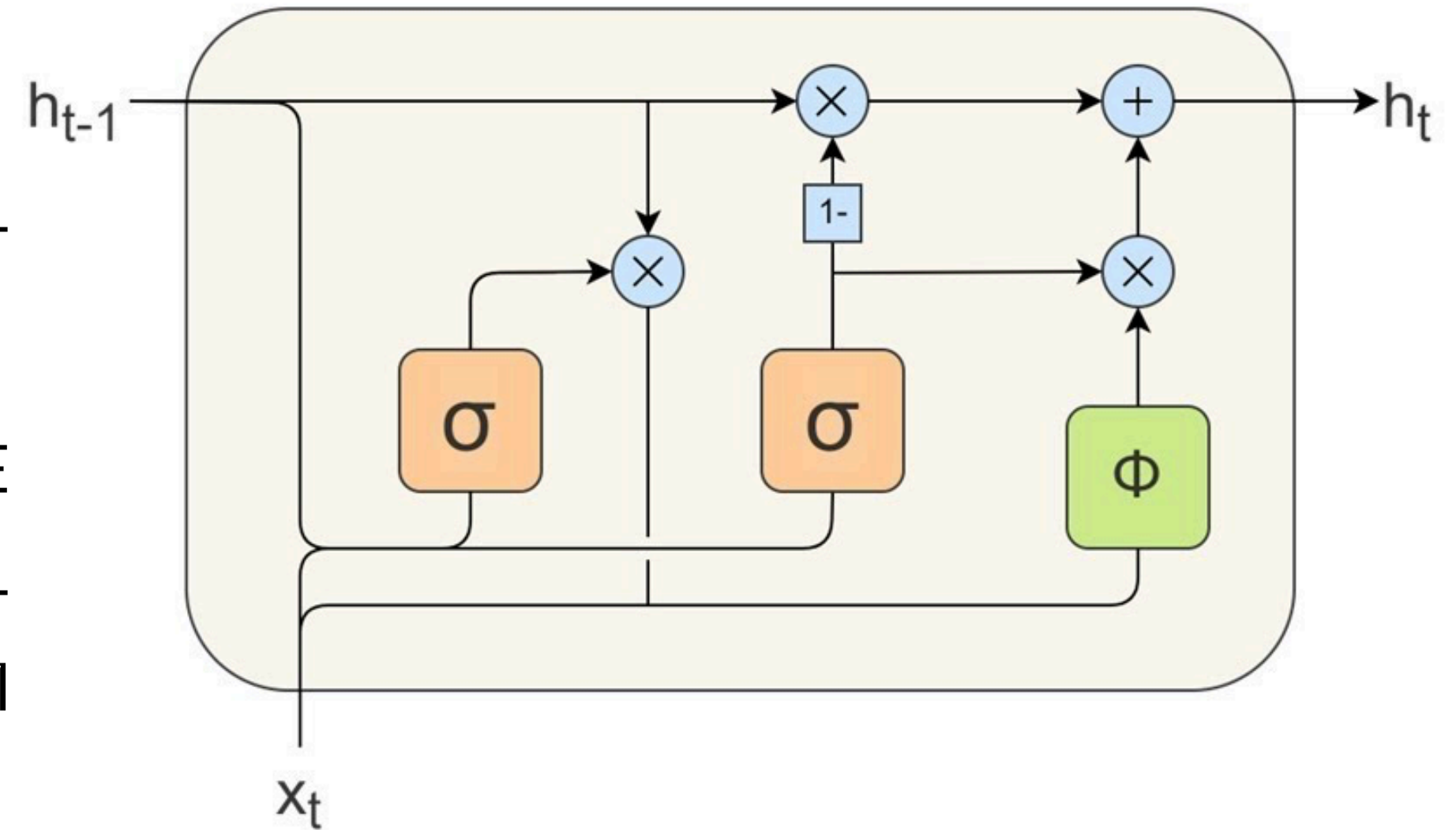
- СИГМА (UPDATE GATE) — РЕШАЕТ: «СКОЛЬКО ОСТАВИТЬ ОТ ПРОШЛОГО?»
- СИГМА (RESET GATE) — РЕШАЕТ: «НАСКОЛЬКО СТЕРЕТЬ СТАРОЕ ПЕРЕД ДОБАВЛЕНИЕМ НОВОГО?»
- Φ (CANDIDATE) — СОЗДАЁТ НОВУЮ ИДЕЮ ИЗ x_t И СТАРОЙ ПАМЯТИ.

СИМВОЛЫ НА СХЕМЕ:

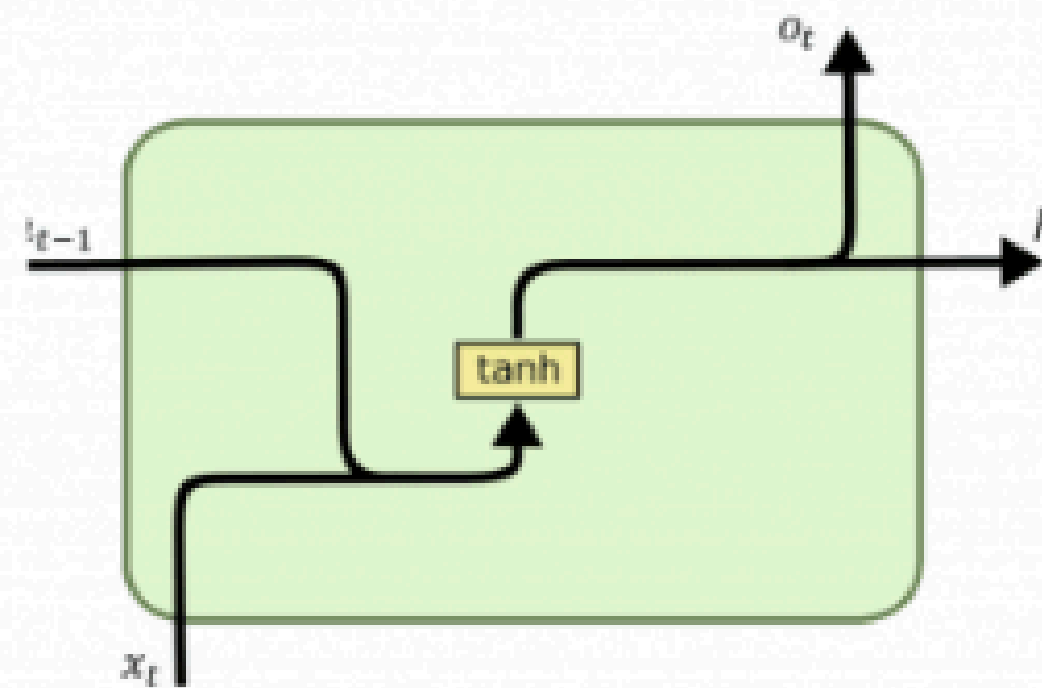
$\times$ — УМНОЖЕНИЕ: "ОСТАВИТЬ ЧАСТЬ"

$1 -$ — "ОБРАТНЫЙ ВЫБОР" (ЧТО НЕ ОСТАВИТЬ)

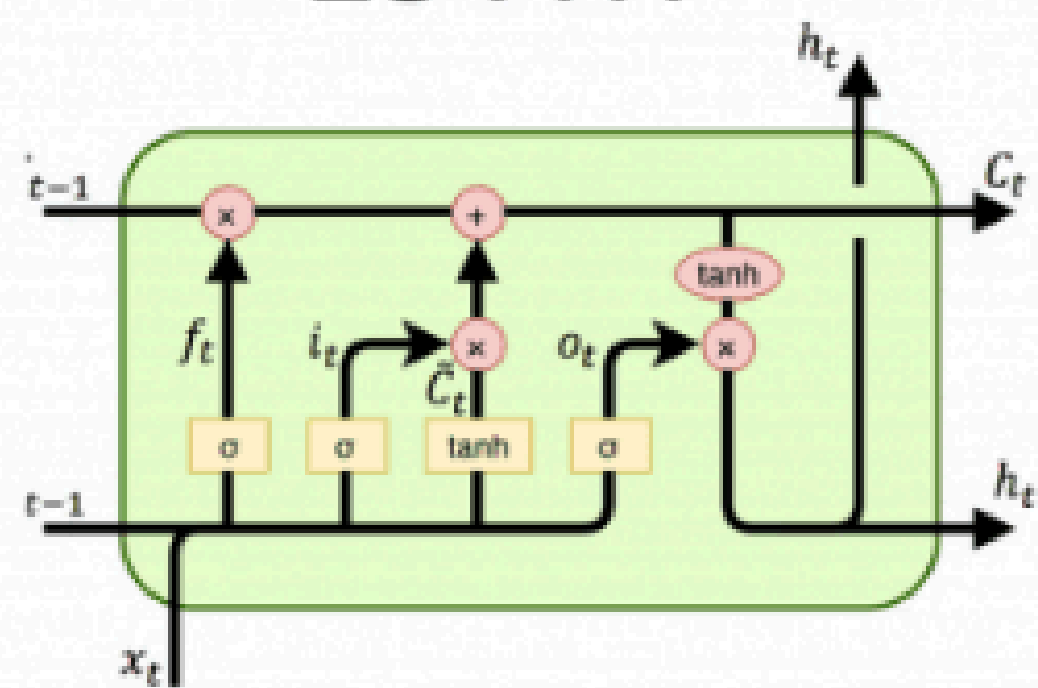
$+$ — СКЛАДЫВАНИЕ СТАРОГО И НОВОГО



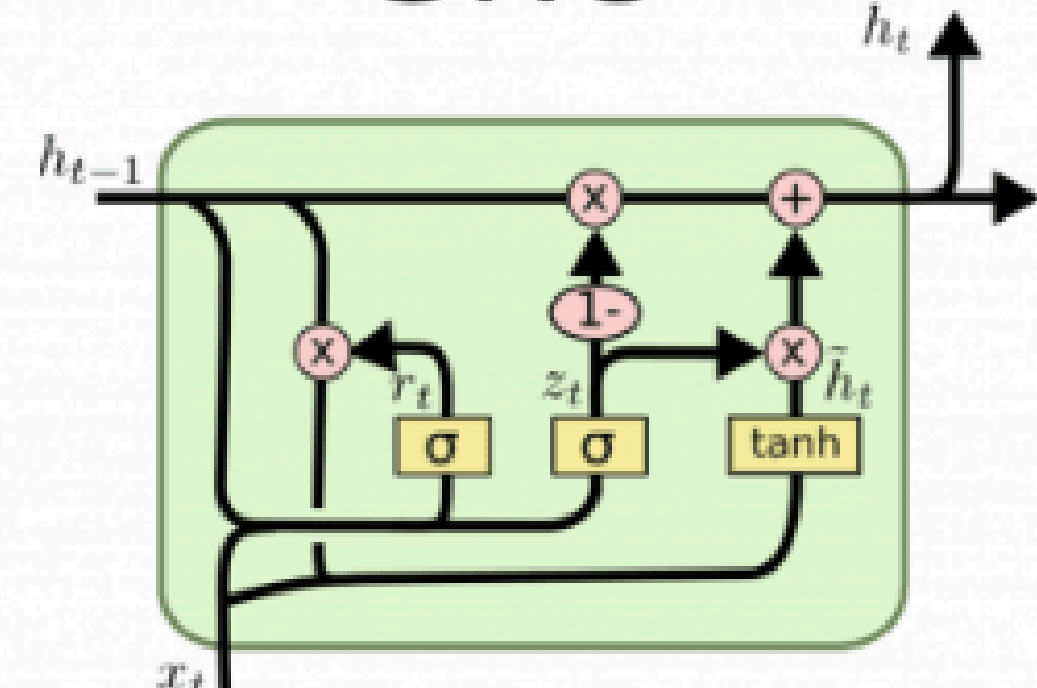
RNN



LSTM



GRU



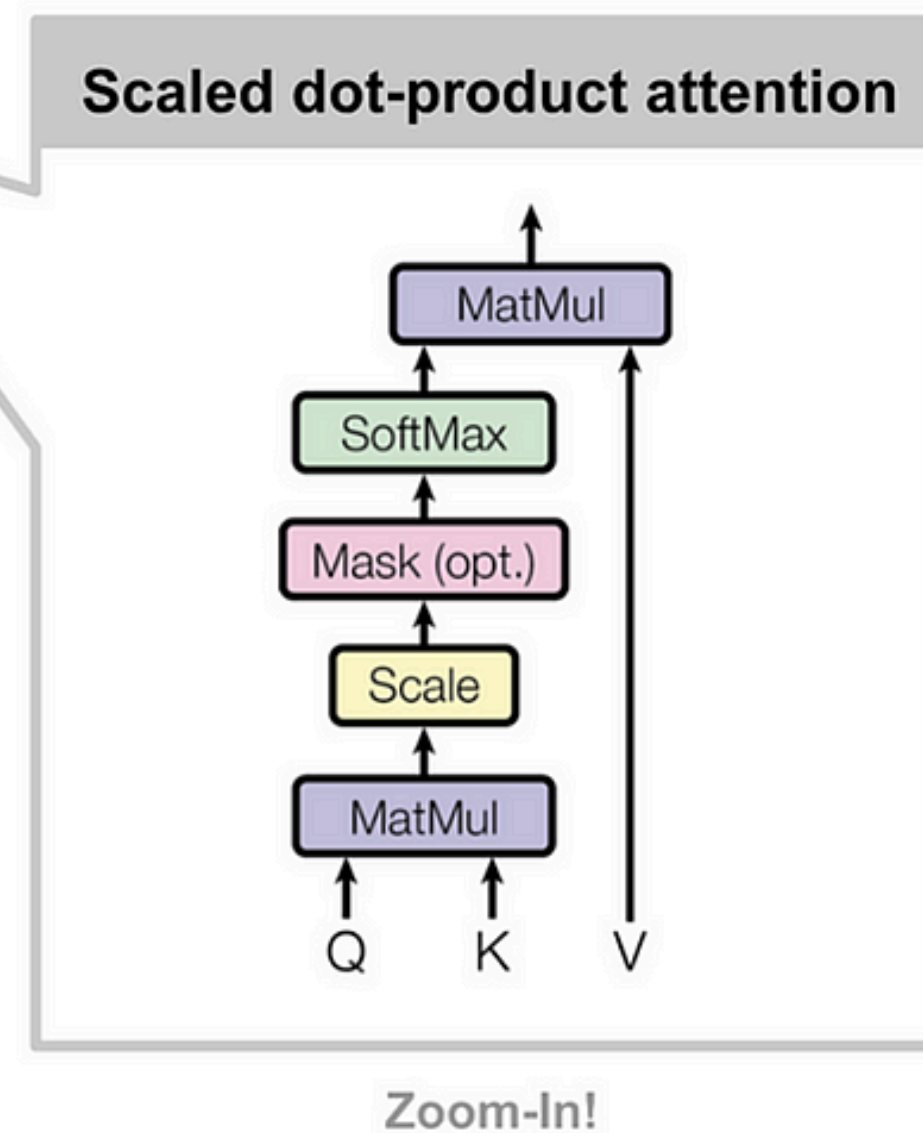
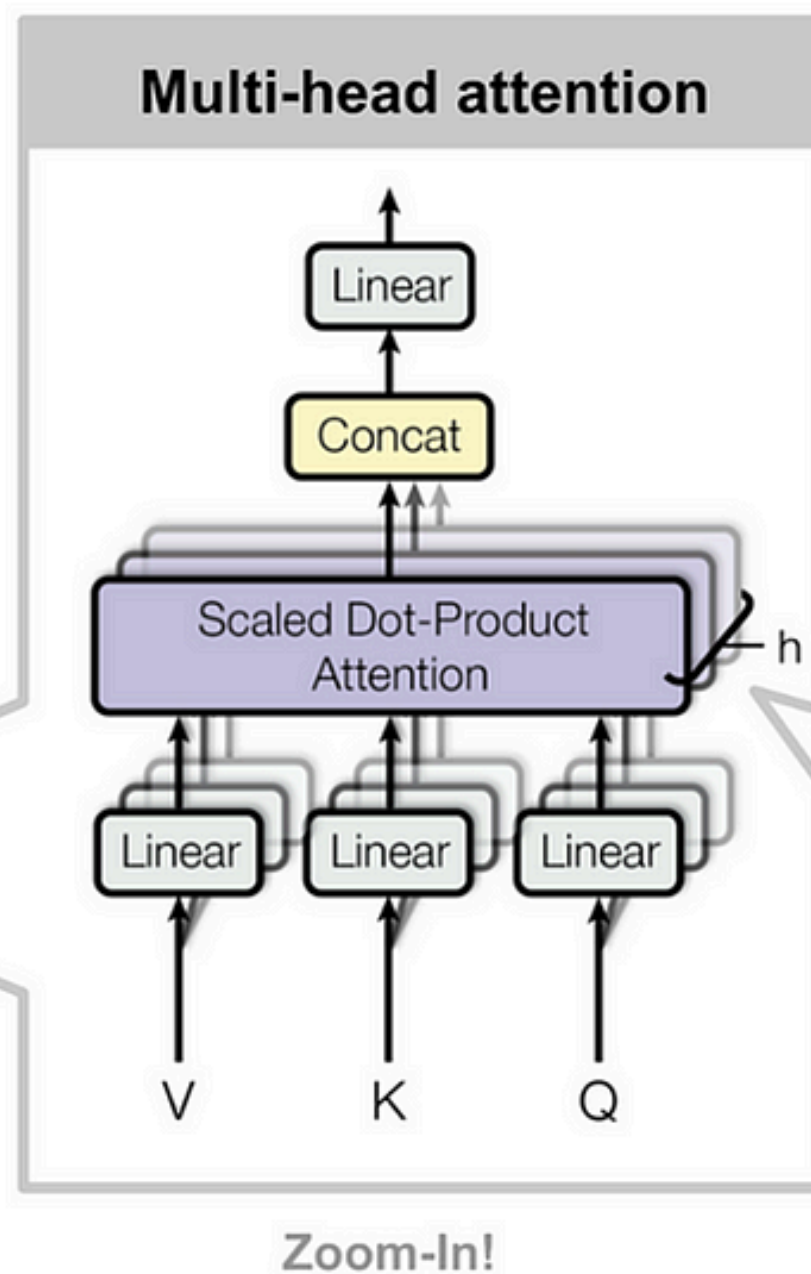
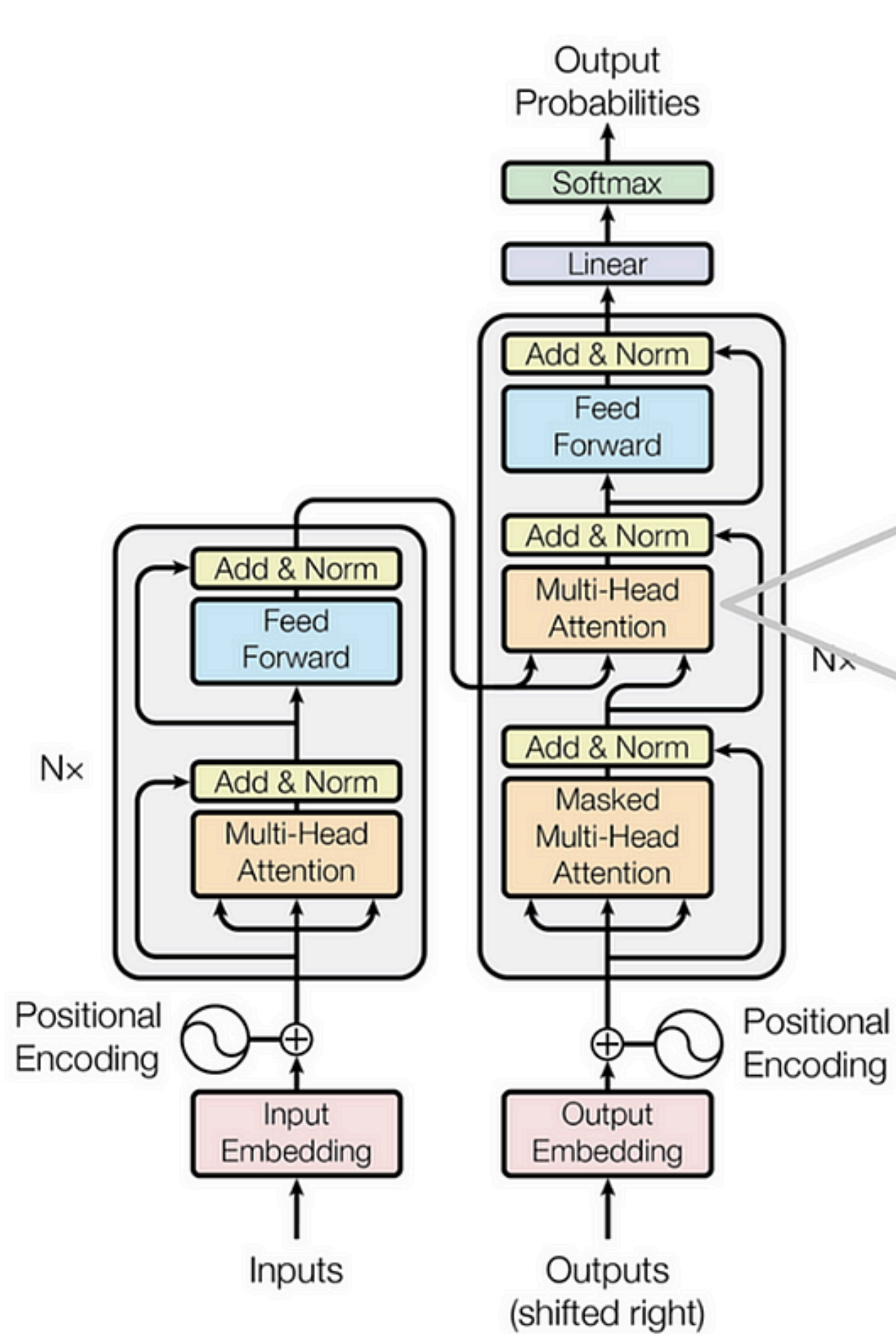
TRANSFORMERS

ПРЕДСТАВЬ СЕБЕ КЛАСС С УЧЕНИКАМИ. КАЖДЫЙ УЧЕНИК СЛУШАЕТ НЕ ТОЛЬКО СЕБЯ, НО И ДРУГИХ — ЧТОБЫ ЛУЧШЕ ПОНЯТЬ, О ЧЁМ ИДЁТ РЕЧЬ.

 ТРАНСФОРМЕР — ЭТО МОДЕЛЬ, КОТОРАЯ ЧИТАЕТ СРАЗУ ВЕСЬ ТЕКСТ, А НЕ ПО СЛОВАМ, КАК RNN.

ОНА СМОТРИТ НА ВСЕ СЛОВА СРАЗУ И РЕШАЕТ, КАКИЕ ВАЖНЕЕ В КАЖДОМ МОМЕНТЕ.

 ГЛАВНАЯ ФИШКА — ВНИМАНИЕ (ATTENTION):
 «НА КАКИЕ СЛОВА СТОИТ ОБРАТИТЬ ВНИМАНИЕ, ЧТОБЫ ЛУЧШЕ ПОНЯТЬ СМЫСЛ?»



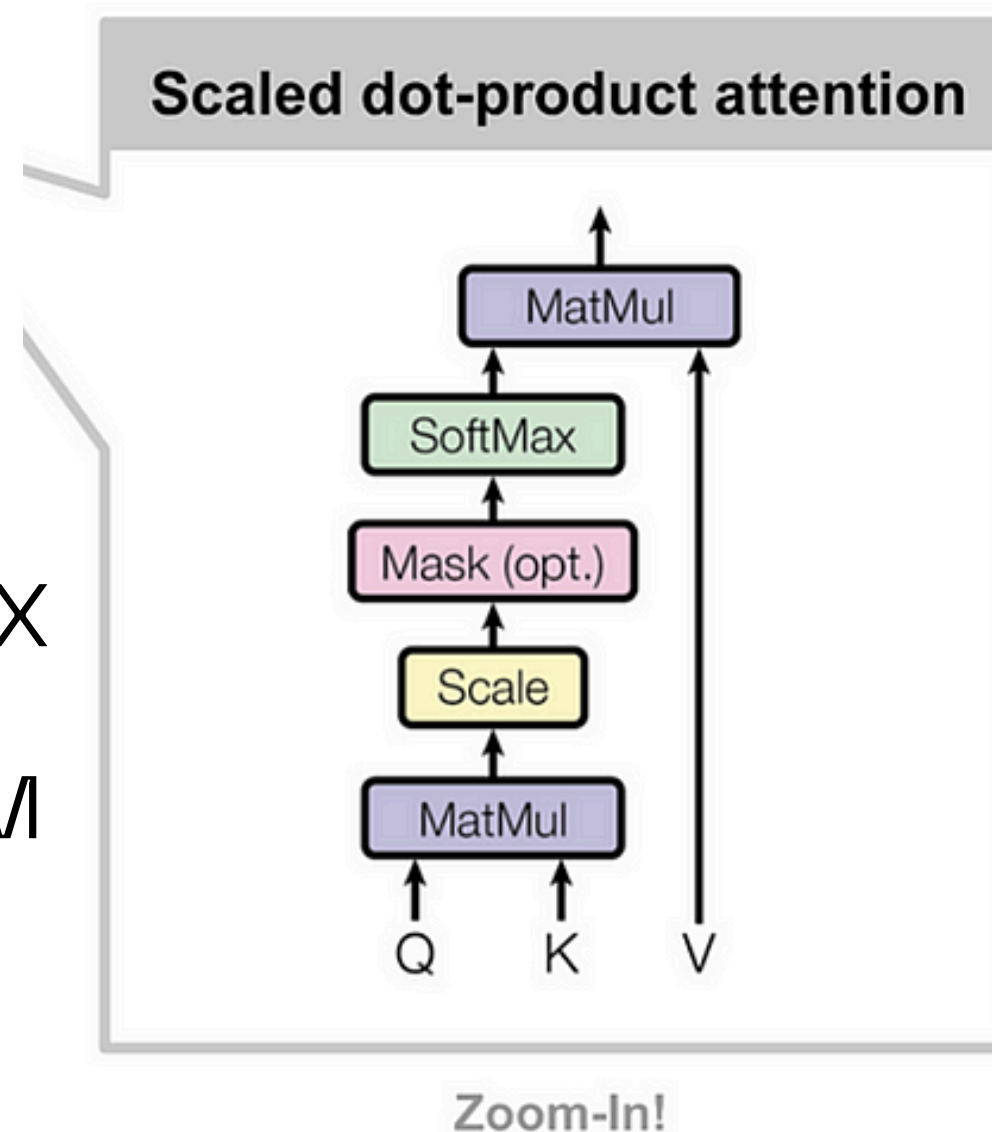
SCALED DOT-PRODUCT ATTENTION

КАЖДОЕ СЛОВО В ПРЕДЛОЖЕНИИ ПОЛУЧАЕТ 3 РОЛИ:

- Q — ВОПРОС: ЧТО Я ИЩУ?
- K — КЛЮЧ: ПОДХОЖУ ЛИ Я ПОД ЧЕЙ-ТО ВОПРОС?
- V — ЗНАЧЕНИЕ: ЧТО Я МОГУ РАССКАЗАТЬ?

🔍 СИСТЕМА СРАВНИВАЕТ ВОПРОСЫ И КЛЮЧИ ВСЕХ СЛОВ → УЗНАЁТ, КТО КОМУ ВАЖЕН.
ЗАТЕМ ОНА СМЕШИВАЕТ ЗНАЧЕНИЯ (V) С НУЖНЫМ ВЕСОМ.

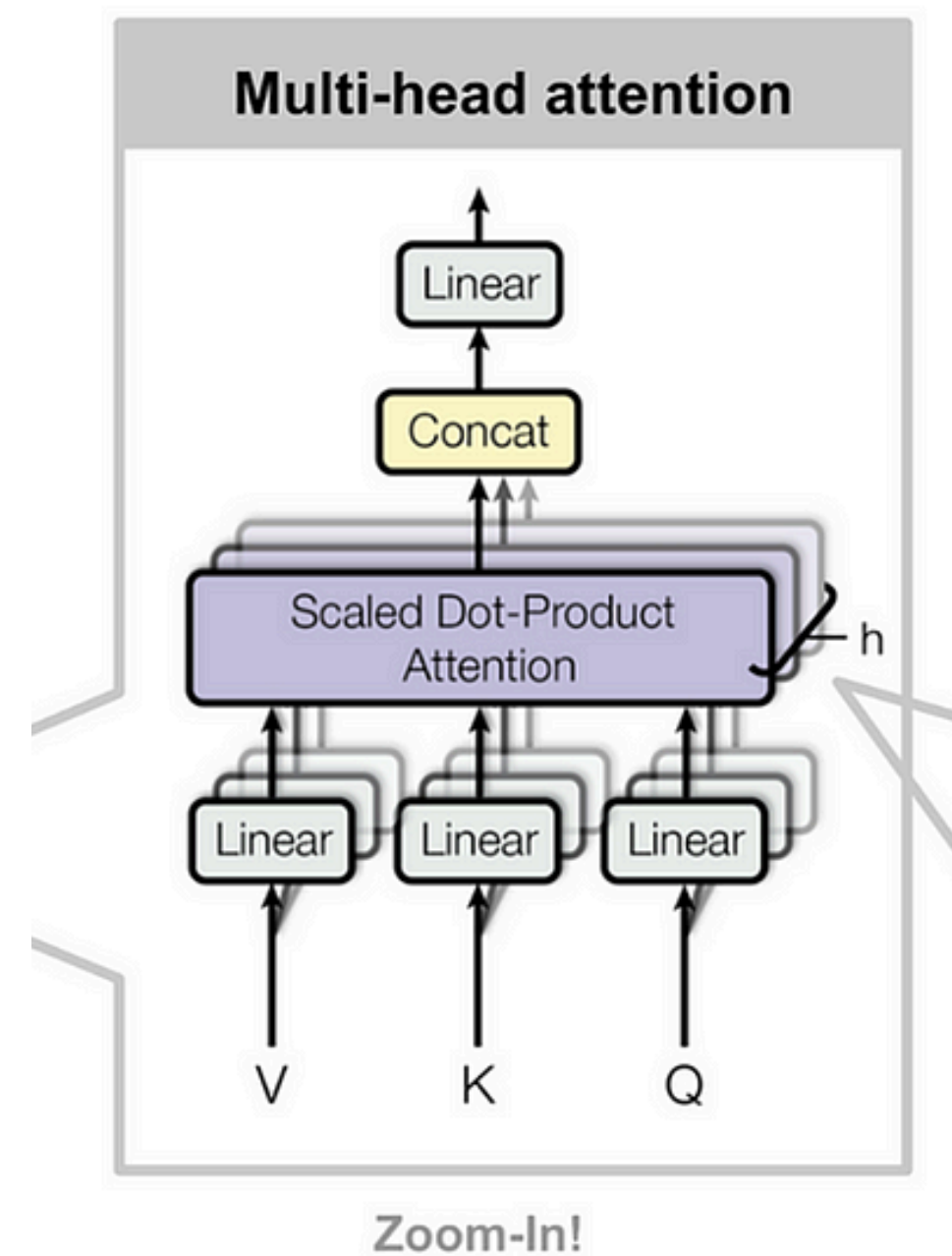
🎯 ЭТО НАЗЫВАЕТСЯ SCALED DOT-PRODUCT ATTENTION — ВЫЧИСЛЕНИЕ ВАЖНОСТИ СЛОВ ДРУГ К ДРУГУ.

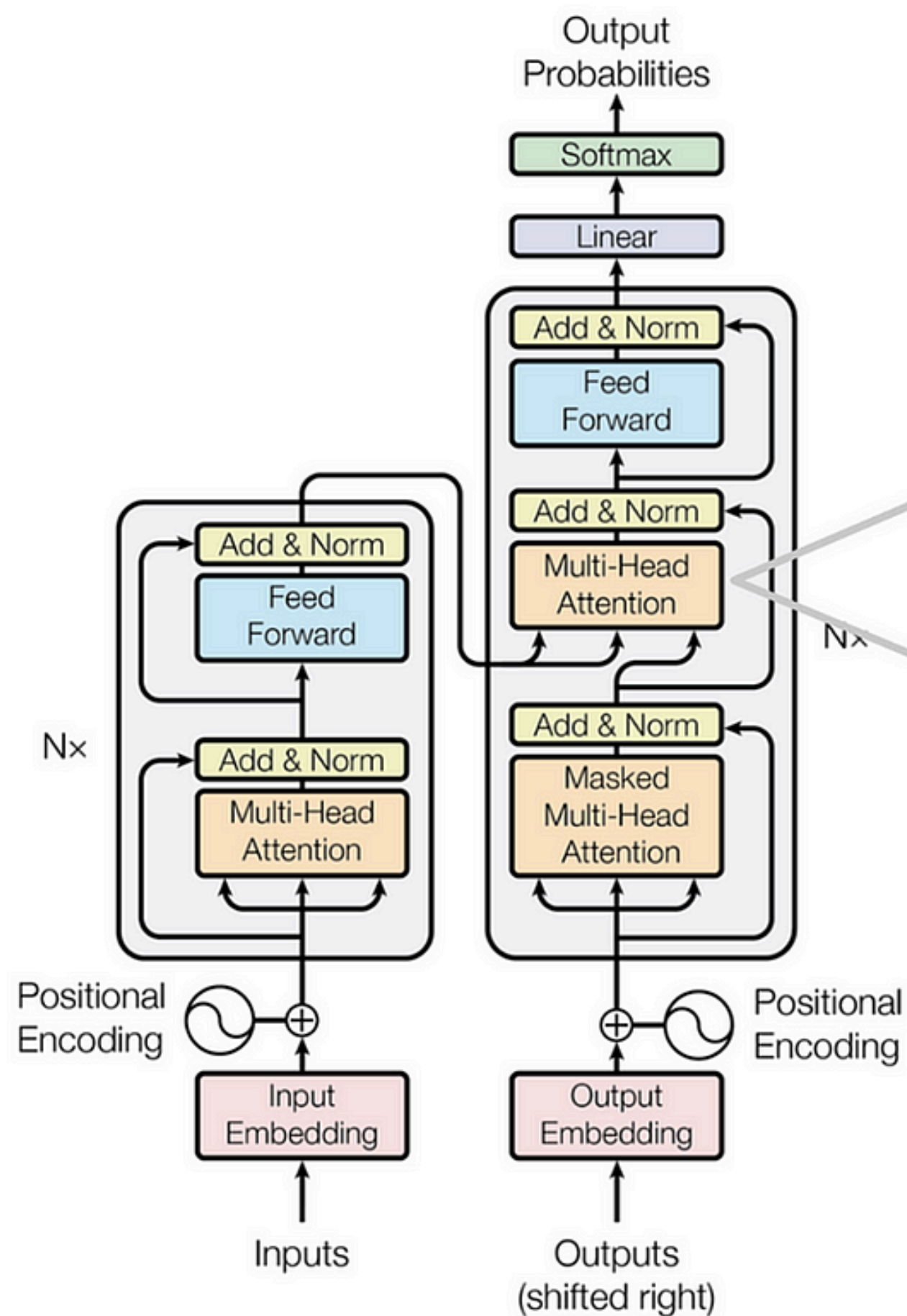


MULTI-HEAD ATTENTION

- 👤 ВМЕСТО ОДНОГО ВНИМАНИЯ — НЕСКОЛЬКО "ГОЛОВ", КАЖДАЯ СМОТРИТ ПО-СВОЕМУ.
- КАЖДАЯ ГОЛОВА ИЗУЧАЕТ ТЕКСТ ПО-СВОЕМУ: ОДНА СМОТРИТ НА ГЛАГОЛЫ, ДРУГАЯ — НА МЕСТОИМЕНЕНИЯ, ТРЕТЬЯ — НА КОНТЕКСТ.
 - ПОТОМ ВСЁ СОБИРАЕТСЯ ВМЕСТЕ (CONCAT) → И ОБРАБАТЫВАЕТСЯ ДАЛЬШЕ.

📌 ЭТО НАЗЫВАЕТСЯ MULTI-HEAD ATTENTION — МОДЕЛЬ ПОЛУЧАЕТ РАЗНЫЕ ТОЧКИ ЗРЕНИЯ СРАЗУ.





ТРАНСФОРМЕР СОСТОИТ ИЗ 2 ЧАСТЕЙ:

🏗 ЭНКОДЕР (СЛЕВА): ЧИТАЕТ ВХОДНОЙ ТЕКСТ И СТРОИТ ЕГО ПОНИМАНИЕ.

🏗 ДЕКОДЕР (СПРАВА): НА ОСНОВЕ ПОНИМАНИЯ ГЕНЕРИРУЕТ ОТВЕТ (НАПРИМЕР, ПЕРЕВОД).

КАЖДЫЙ СЛОЙ ВКЛЮЧАЕТ:

💡 ВНИМАНИЕ (SELF-ATTENTION),
 + СЛОЖЕНИЕ + НОРМАЛИЗАЦИЯ (ADD & NORM),

🔄 ПОЛНОСВЯЗНУЮ СЕТЬ (FEED FORWARD),

🔄 ПОВТОРЯЕТСЯ МНОГО РАЗ (N РАЗ).

📌 В КОНЦЕ — SOFTMAX, КОТОРЫЙ ВЫБИРАЕТ ИТОГОВОЕ СЛОВО.

TRANSFORMERS

ЕСТЬ РАЗНЫЕ ТИПЫ ТРАНСФОРМЕРОВ — ВОТ САМЫЕ ИЗВЕСТНЫЕ:

🧱 ENCODER-ONLY
ПОНИМАЕТ ВХОД, НЕ ГЕНЕРИРУЕТ
BERT, ROBERTA

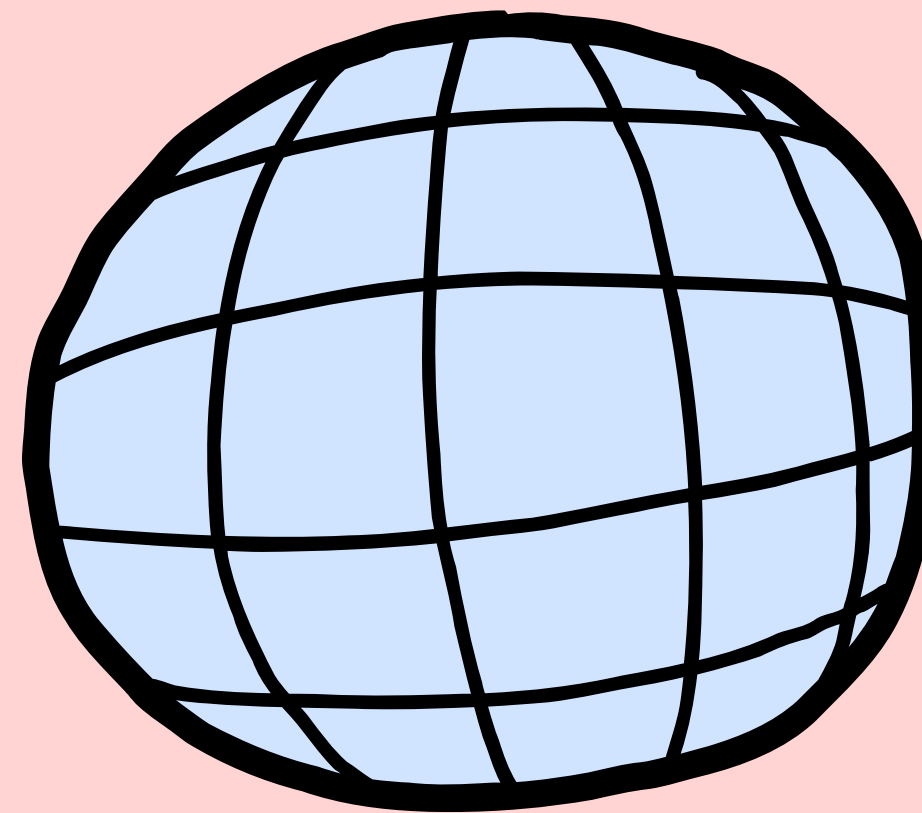
🧱 DECODER-ONLY
ГЕНЕРИРУЕТ НА ОСНОВЕ ПРЕДЫДУЩЕГО
GPT, LLAMA, FALCON

🧱 ENCODER-DECODER
ПЕРЕВОДИТ ИЛИ СОЗДАЁТ НА ОСНОВЕ ВХОДА
T5, BART, MT5



ПОЛЕЗНЫЕ ССЫЛКИ

- Курс Лены Войты
- Как на самом деле работает механизм внимания
- Домашки ШАДа
- Вопросы с собеседования на NLP engineer
- Regression with Text Input Using BERT and Transformers
- Датасет чтобы потренироваться в сентимент анализе



ССЫЛКА НА ЭТУ ПРЕЗЕНТАЦИЮ

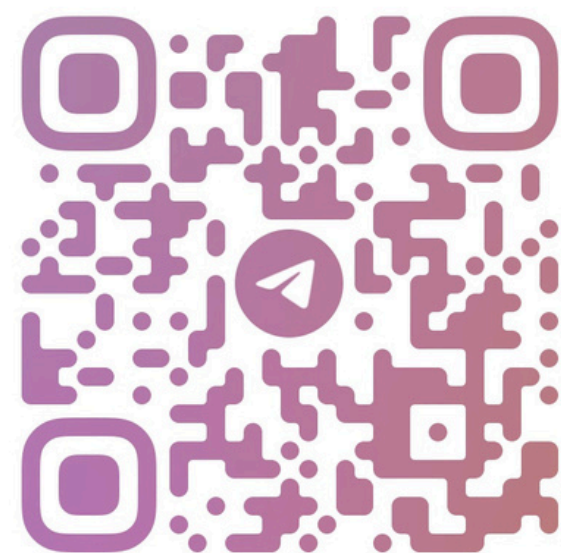


ссылка на ноутбук, в
котором можно и нужно
баловаться

КОНТАКТЫ

это Катя, автор презентации

это Катин телеграм, на случай
если появятся вопросы или
предложения по материалу



@KDANKOVA

